

PROJET DE FIN D'ÉTUDES

GEOCRAS

Géolocalisation de garages et assistance à la conduite

Cahier des charges et dossier de conception
Application mobile pour le marché camerounais

Yaoundé, Cameroun
Août 2026

Version 1.0

Résumé

Tomber en panne à Yaoundé se règle aujourd'hui au téléphone et de mémoire. On appelle un proche, qui appelle un mécanicien qu'il connaît, et l'on attend au bord de la route sans savoir si quelqu'un viendra ni quand. Le temps d'attente n'est pas mesuré, il est subi. GeoCras part de cette situation et propose un service qui la remplace point par point : une carte des garages autour de soi, classés par pertinence réelle et non par ordre alphabétique ; une déclaration de panne qui tient en trois écrans et part avec la position ; un suivi en direct du dépanneur jusqu'à ce qu'il arrive ; et une trace horodatée de l'intervention qui, une fois les deux parties d'accord sur le fait qu'elles se sont rejointes, crédite des points de fidélité convertibles en remises.

Le document qui suit est le cahier des charges de ce service et le dossier de conception de sa première version. Il commence par une analyse du terrain et une critique argumentée des solutions existantes, formelles comme informelles. Il expose ensuite la solution retenue, ses spécifications générales puis détaillées — modélisation UML complète, schéma de base de données, contrats d'interface — et justifie chaque choix technique par un raisonnement rattaché aux contraintes du contexte camerounais : précision GPS médiocre en centre-ville, réseau mobile intermittent, forfaits de données comptés, et une chaîne de fidélité qui manipule ce qui est en pratique de l'argent.

La dernière partie rend compte de ce qui a été effectivement construit. L'application existe, elle compte environ quarante-sept mille lignes de TypeScript réparties entre un client mobile React Native, une interface de programmation Express et un paquet de contrats partagés, avec quarante-quatre routes exposées, quatorze tables et cent trente-trois tests automatisés. Le document décrit les interfaces écran par écran, expose les mesures obtenues et énonce sans détour ce qui reste à faire.

Abstract

Breaking down in Yaoundé is currently handled by phone and from memory. You call a relative, who calls a mechanic they know, and you wait by the roadside with no way of knowing whether anyone is coming, or when. GeoCras replaces that sequence with a map of nearby garages ranked by genuine relevance, a breakdown report that fits in three screens and carries the user's position, live tracking of the mechanic until arrival, and a timestamped record of the job which — once both parties agree they have met — credits loyalty points convertible into discounts at certified garages.

This document is both the requirements specification and the design record for the first version. It analyses the field and criticises existing alternatives, formal and informal alike; sets out the proposed solution, its general and then detailed specifications including full UML

modelling, the database schema and the interface contracts; and justifies every technical decision against the constraints of the Cameroonian context: poor GPS accuracy downtown, intermittent mobile coverage, metered data plans, and a loyalty chain that handles what is effectively cash. The final part reports on what was actually built — roughly forty-seven thousand lines of TypeScript, forty-four routes, fourteen tables and one hundred and thirty-three automated tests — and states plainly what remains to be done.

Sommaire

Résumé	2
Abstract	2
1 Introduction	7
1.1 Contexte	7
1.2 Problématique	7
1.3 Objectifs	8
1.4 Méthodologie et lecture du document	8
2 Analyse des besoins et étude de l'existant	10
2.1 Le terrain	10
2.2 Étude de l'existant	10
2.2.1 Les cartes généralistes	10
2.2.2 Les annuaires et les pages de réseaux sociaux	11
2.2.3 L'assistance contractuelle des assureurs	11
2.2.4 Les applications de mise en relation à la demande	11
2.3 Critique de l'existant	12
2.4 Besoins fonctionnels	13
2.5 Besoins non fonctionnels	13
2.6 Les acteurs	15
3 La solution proposée	16
3.1 Principe	16
3.2 Ce qui distingue la solution	16
3.3 Périmètre de la première version	17
3.4 Les trois risques identifiés	17
4 Spécifications générales	19
4.1 Architecture générale du système	19
4.2 Acteurs et droits	20
4.3 Diagramme de cas d'utilisation	21
4.4 Description des cas d'utilisation principaux	22
4.4.1 Déclarer une panne	22
4.4.2 Suivre l'intervention	22
4.4.3 Confirmer l'arrivée	23
5 Spécifications détaillées	24

5.1	Diagramme de classes-----	24
5.2	Diagramme d'états-transitions de la demande d'assistance-----	25
5.3	Diagramme de séquence du parcours d'assistance-----	27
5.4	Diagramme de séquence du mode dégradé-----	28
5.5	Diagramme d'activité de la déclaration de panne-----	30
5.6	Schéma de la base de données-----	31
5.6.1	Les règles métier inscrites comme contraintes-----	32
5.6.2	La requête de pertinence-----	33
5.7	Contrats de l'interface de programmation-----	34
5.8	Protocole temps réel-----	35
5.9	Dispositif anti-fraude-----	36
6	Choix techniques et justifications-----	38
6.1	Monorepo et contrats partagés-----	38
6.2	PostgreSQL avec extension géospatiale-----	38
6.3	Constructeur de requêtes typé plutôt qu'ORM complet-----	39
6.4	Rendu cartographique-----	39
6.5	Canal temps réel-----	40
6.6	React Native et Expo-----	40
6.7	Répartition de l'état côté mobile-----	41
6.8	Stockage des photographies-----	41
6.9	Synthèse des choix-----	42
7	Résultats obtenus-----	43
7.1	État d'avancement-----	43
7.2	Les interfaces-----	44
7.2.1	Lancement et carte principale-----	44
7.2.2	Déclaration de panne et résultats-----	45
7.2.3	Itinéraire, suivi et avis-----	46
7.2.4	Mode conduite-----	47
7.2.5	Fidélité, menu et paramètres-----	48
7.3	Tests et validation-----	49
7.4	Ce qui reste à faire-----	50
8	Conclusion-----	51
8.1	Bilan-----	51
8.2	Limites-----	51
8.3	Perspectives-----	52
8.4	Références-----	52

Table des figures

Figure 1	Architecture générale : couches, responsabilités et dépendances externes-----	19
Figure 2	Diagramme de cas d'utilisation : acteurs, paquets fonctionnels et relations d'inclusion et d'extension-----	21
Figure 3	Diagramme de classes du domaine : entités, attributs, opérations et cardinalités-----	24
Figure 4	Cycle de vie d'une demande d'assistance, avec la garde inscrite en base-----	26
Figure 5	Séquence complète d'une intervention, de la déclaration de panne à la clôture-----	28
Figure 6	Perte de connexion, bascule en mode dégradé et rattrapage par rejeu du journal-----	29
Figure 7	Activité de déclaration de panne, réparties entre le client, l'application et le serveur-----	30
Figure 8	Modèle physique de données : quatorze tables, clés, contraintes et index-----	31
Figure 9	Écran de lancement avec acquisition de la position, et carte principale avec marqueurs numérotés-----	44
Figure 10	Déclaration de panne avec tri automatique des problèmes, et résultats classés avec fiche contextuelle-----	45
Figure 11	Suivi bidirectionnel avec double estimation d'arrivée, et fiche complète d'un garage avec ses avis-----	46
Figure 12	Mode conduite au repos et en session, avec vitesse, statistiques et pile d'alertes-----	47
Figure 13	Espace de fidélité avec grade et progression, et tiroir latéral de navigation-----	48
Figure 14	Écran de paramètres en mode sombre-----	49

Introduction

1.1 Contexte

Le parc automobile camerounais est vieillissant, majoritairement composé de véhicules importés d'occasion, et il circule sur un réseau routier dont une partie notable n'est pas bitumée. La conséquence est mécanique au sens propre : la panne n'est pas un incident rare, c'est un événement ordinaire de la vie d'un conducteur. Batterie qui lâche, pneu crevé sur un nid-de-poule, surchauffe dans un embouteillage de fin d'après-midi, panne sèche parce que la jauge ment depuis deux ans. À Yaoundé, ville de collines où les rues secondaires se ressemblent et où les adresses formelles sont rares, un véhicule immobilisé pose immédiatement deux problèmes distincts : trouver quelqu'un capable d'intervenir, et lui expliquer où l'on se trouve.

Le second problème est souvent le plus coûteux. Décrire sa position par téléphone, à quelqu'un qui ne connaît pas le quartier, en s'appuyant sur des repères — « après la station, en face du bar bleu, avant le carrefour » — prend du temps, produit des malentendus, et se solde régulièrement par un dépanneur qui tourne pendant vingt minutes à trois cents mètres de son client. Or ce temps est exactement celui pendant lequel le conducteur est arrêté sur une chaussée, parfois de nuit, parfois avec des passagers, parfois en un endroit où il préférerait ne pas rester.

Dans le même temps, le téléphone que ce conducteur a dans la poche connaît sa position à quelques dizaines de mètres près, sait la transmettre, et pourrait la montrer à un garagiste sur une carte. C'est un décalage frappant entre la technologie disponible et la manière dont le service est réellement rendu. GeoCras existe pour combler ce décalage.

1.2 Problématique

La question à laquelle ce projet répond peut se formuler en une phrase : comment réduire le délai et l'incertitude entre le moment où un véhicule s'immobilise et le moment où un professionnel compétent se trouve à côté de lui ?

Cette formulation contient déjà trois exigences qui ne sont pas interchangeables. Le délai suppose que l'on trouve vite, donc que le classement des garages proposés soit pertinent et non arbitraire. L'incertitude suppose que l'on sache où en est la demande à tout instant, donc un suivi et non une simple mise en relation. Et la compétence suppose que l'on puisse distinguer un atelier sérieux d'un atelier quelconque, donc un mécanisme de confiance qui résiste à la manipulation.

S'y ajoute une contrainte de contexte qui traverse tout le projet et qui interdit un certain nombre de solutions apparemment évidentes : le réseau mobile camerounais est irrégulier, la

couverture passe de la 4G à la 2G en changeant de rue, les coupures sont fréquentes, et les forfaits de données se comptent. Une application qui suppose une connexion permanente et abondante ne fonctionnera pas ici. Une application dont la fonction principale est de servir quelqu'un en difficulté au bord d'une route doit continuer de rendre un service utile quand le réseau se dégrade, pas afficher une roue qui tourne.

1.3 Objectifs

L'objectif général est de livrer une application mobile utilisable en situation réelle de panne, sur un téléphone Android de milieu de gamme, à Yaoundé, avec un réseau imparfait.

Les objectifs particuliers découlent de la problématique. Il s'agit d'abord de capter automatiquement la position du conducteur au lancement, de manière à ce que la déclaration de panne n'ait jamais à contenir une adresse tapée à la main. Il s'agit ensuite de présenter les garages environnants sur une carte, numérotés selon un classement calculé par le serveur et modifiable par l'utilisateur — le plus proche, le mieux noté, le certifié — de façon à ce que le numéro affiché ait un sens et reste cohérent quel que soit l'écran où on le regarde. Il s'agit également d'établir un suivi bidirectionnel après l'appel, avec un temps d'arrivée estimé pour chacune des deux parties, recalculé à partir des déplacements réels et non d'une distance à vol d'oiseau figée.

Il s'agit enfin de construire une chaîne de confiance qui tienne : une intervention n'est réputée terminée que lorsque le client et le garagiste ont chacun confirmé leur arrivée, et c'est cette double confirmation qui ouvre le crédit de points de fidélité. Comme ces points ont vocation à devenir des remises puis, à terme, un équivalent en Mobile Money, ils constituent une valeur monétaire, et le système doit être conçu dès le premier jour pour résister à quelqu'un qui chercherait à en fabriquer.

En dehors de l'assistance proprement dite, l'application embarque un mode conduite qui affiche la vitesse, la distance et des alertes pendant un trajet. Cette partie est explicitement une simulation dans la première version : aucun capteur réel n'est branché. L'objectif la concernant n'est donc pas la véracité des alertes, mais la qualité de l'architecture qui les produit, de manière à pouvoir substituer une source réelle sans réécrire l'interface.

1.4 Méthodologie et lecture du document

Le projet a été conduit en partant des écrans plutôt que des tables. Onze maquettes ont été dessinées avant la première ligne de code, et elles ont servi de référence tout au long du développement : lorsque le code et l'idée que l'on s'en faisait divergeaient, ce sont les maquettes qui faisaient foi. Cette contrainte a eu un effet direct sur la modélisation, et le document le signale à plusieurs reprises — la numérotation des marqueurs, par exemple, est une exigence d'interface qui a imposé un choix de requête SQL.

Le développement lui-même a suivi un ordre dicté par le risque : ce qui pouvait faire échouer le projet a été traité en premier, et ce qui pouvait attendre a attendu. Les trois risques identifiés au démarrage sont exposés au chapitre 3 et leur traitement est repris au chapitre 6.

La suite du document se lit ainsi. Le chapitre 2 analyse les besoins et critique l'existant. Le chapitre 3 expose la solution retenue et son périmètre. Le chapitre 4 en donne les spécifications générales, acteurs et cas d'utilisation. Le chapitre 5 descend au niveau des spécifications détaillées : diagrammes de classes, d'états-transitions, de séquence et d'activité, schéma de base de données, contrats d'interface. Le chapitre 6 justifie les choix techniques un par un. Le chapitre 7 rend compte des résultats obtenus, interfaces à l'appui. Le chapitre 8 conclut et ouvre sur la suite.

Analyse des besoins et étude de l'existant

2.1 Le terrain

Avant de comparer des solutions, il faut décrire honnêtement ce qui se passe aujourd'hui quand un véhicule s'arrête. Le scénario dominant, et de très loin, n'est pas technologique.

Le conducteur immobilisé appelle quelqu'un qu'il connaît : un ami, un frère, un collègue qui « connaît un bon mécanicien ». Cette personne relaie l'appel vers un garagiste de son réseau. Le garagiste, s'il est disponible, demande où se trouve le véhicule. Suit une conversation d'orientation qui peut durer plusieurs minutes et qui repose entièrement sur des repères partagés — un carrefour, une enseigne, une station-service. Puis le conducteur attend, sans information, jusqu'à ce que quelqu'un arrive ou n'arrive pas. S'il rappelle, il obtient une estimation dont la fiabilité est celle de la politesse.

Ce circuit a des qualités qu'il serait malhonnête de nier. Il repose sur la confiance interpersonnelle, qui est un mécanisme puissant : le mécanicien recommandé par un proche a une réputation à défendre auprès de ce proche. Il est gratuit, immédiat, et ne suppose ni application installée ni forfait de données. Il fonctionne aussi bien à trois heures du matin qu'à midi.

Mais il a trois défauts structurels. Il ne fonctionne que si l'on a le bon contact, ce qui exclut les personnes de passage, les nouveaux arrivants, et tous ceux dont le réseau ne compte pas de garagiste. Il ne donne aucune visibilité : ni sur le délai, ni sur le prix, ni sur le fait que quelqu'un est effectivement en route. Et il ne laisse aucune trace : la même personne recommencera intégralement la prochaine fois, sans que rien de ce qui a été appris ne soit conservé.

2.2 Étude de l'existant

2.2.1 Les cartes généralistes

Un conducteur qui cherche un garage sur une carte généraliste obtient une liste de points d'intérêt et, souvent, un numéro de téléphone. C'est déjà mieux que rien : la position est connue, l'itinéraire est calculé, et l'application est déjà installée sur la plupart des téléphones.

La limite tient à la nature des données. Un point d'intérêt sur une carte généraliste porte un nom, une catégorie et parfois des horaires. Il ne porte ni la spécialité du garage, ni le fait qu'il dispose ou non d'un véhicule de remorquage, ni sa capacité à se déplacer, ni aucune certification. La couverture des ateliers de quartier y est par ailleurs très inégale, et les horaires affichés sont fréquemment obsolètes. Surtout, la relation s'arrête au numéro de

téléphone : la carte ne sait pas qu'un dépannage est en cours, ne suit personne, et n'enregistre rien.

Il faut y ajouter un point souvent négligé : la donnée d'un garage — sa certification, sa note, ses photos, ses horaires réels, sa disponibilité — est une donnée métier propriétaire. Elle n'a aucune raison d'exister dans une base cartographique généraliste, et elle ne s'y trouvera jamais. C'est précisément cette donnée que GeoCras doit produire et détenir.

2.2.2 Les annuaires et les pages de réseaux sociaux

Beaucoup de garages camerounais existent en ligne sous la forme d'une page de réseau social ou d'une fiche d'annuaire. On y trouve un numéro, parfois des photos, parfois un quartier.

Ces supports règlent partiellement le problème de la découverte mais aucun des autres. Ils ne sont pas géolocalisés au sens strict — un quartier n'est pas une position —, ils ne permettent aucun classement par distance réelle, et leur fraîcheur n'est garantie par personne. Un numéro qui ne répond plus reste affiché indéfiniment. Pour quelqu'un en panne, une liste de numéros dont on ignore lesquels sont encore valides n'est pas une réponse.

2.2.3 L'assistance contractuelle des assureurs

Certaines polices d'assurance incluent une garantie d'assistance, avec un numéro à appeler et un prestataire qui se déplace. C'est la solution la plus proche d'un service structuré, et elle apporte une vraie garantie : quelqu'un est contractuellement tenu de venir.

Trois limites en réduisent la portée. La première est l'accès : cette garantie suppose une police qui l'inclut, ce qui restreint le service à une fraction du parc. La deuxième est le fonctionnement, qui reste téléphonique et centralisé, avec les mêmes difficultés de localisation que le circuit informel — l'opérateur qui prend l'appel n'est pas sur place et doit reconstituer la position par la parole. La troisième est l'absence de choix : l'assuré ne choisit pas son garage, il reçoit celui que le prestataire mandate, sans visibilité sur sa note ni sur ses spécialités.

2.2.4 Les applications de mise en relation à la demande

Le modèle des applications de transport à la demande est le plus proche parent conceptuel de GeoCras : géolocalisation, mise en relation, suivi en temps réel, notation. Là où ces applications existent, elles démontrent que le schéma fonctionne et que les utilisateurs l'adoptent.

La transposition directe au dépannage se heurte cependant à des différences réelles. Une course de transport est un service homogène : n'importe quel véhicule conforme peut l'exécuter. Un dépannage ne l'est pas : une panne de boîte de vitesses sur un camion et une batterie à plat sur une moto n'appellent ni les mêmes compétences, ni le même outillage, ni le même véhicule d'intervention. Le classement des prestataires ne peut donc pas se réduire à la distance. Par ailleurs, la course de transport se termine par un dépôt à une adresse,

événement objectif et vérifiable ; l'intervention de dépannage se termine par une rencontre, ce qui pose un problème de preuve entièrement différent et bien plus difficile – problème que le chapitre 5 traite en détail.

2.3 Critique de l'existant

Le tableau suivant résume la comparaison sur les critères qui comptent pour un conducteur immobilisé.

Tableau 1 — Comparaison des solutions existantes sur les besoins d'un conducteur immobilisé

Critère	Réseau personnel	Carte généraliste	Annuaire en ligne	Assistance assureur	GeoCras
Accessible sans contact préalable	Non	Oui	Oui	Réservé aux assurés	Oui
Position transmise automatiquement	Non	Partiellement	Non	Non	Oui
Classement par pertinence réelle	Non	Par distance seule	Non	Sans choix	Distance, note, certification
Spécialité et outillage connus	Par la confiance	Non	Rarement	Par le prestataire	Oui, par service déclaré
Suivi du dépanneur en direct	Non	Non	Non	Non	Oui, dans les deux sens
Temps d'arrivée estimé	Non	Trajet théorique	Non	Annoncé oralement	Calculé, recalculé, partagé
Trace de l'intervention	Non	Non	Non	Chez l'assureur	Historique des deux parties
Fonctionne en réseau dégradé	Oui	Partiellement	Partiellement	Oui	Oui, mode dégradé prévu
Coût pour l'utilisateur	Nul	Nul	Nul	Inclus dans la prime	Nul, points gagnés

La lecture de ce tableau fait apparaître un constat qui a orienté tout le projet : aucune solution existante ne couvre simultanément la découverte, le classement pertinent, le suivi et la trace. Chacune en couvre une ou deux. Le réseau personnel est imbattable sur la confiance et l'accessibilité hors ligne, mais il ne donne ni visibilité ni mémoire. Les cartes généralistes donnent la position mais s'arrêtent au numéro de téléphone. L'assistance contractuelle donne une garantie mais retire le choix et reste aveugle sur la localisation.

Il faut en tirer une conséquence de conception, et non seulement un argument commercial : GeoCras ne remplace pas le téléphone, il l'outille. Le bouton d'appel reste présent sur chaque fiche de garage, et l'appel téléphonique demeure le moyen par lequel le client et le garagiste se parlent réellement. Ce que l'application apporte, c'est ce qui manque autour de cet appel — la position transmise sans être décrite, le classement qui a du sens, le suivi qui remplace le rappel, et la trace qui permet à la relation de se répéter.

2.4 Besoins fonctionnels

Les besoins fonctionnels se répartissent en six ensembles cohérents, qui correspondent aux paquets du diagramme de cas d'utilisation présenté au chapitre 4.

La gestion du compte couvre l'inscription et la connexion — par numéro de téléphone, qui est l'identifiant naturel au Cameroun, plutôt que par adresse électronique —, la gestion des véhicules enregistrés destinés à préremplir les demandes futures, et la modification du profil.

La recherche de garages couvre la capture automatique de la position au lancement, la recherche des garages autour de cette position dans un rayon paramétrable, le tri au choix par distance, par note ou par certification, le filtrage par service offert et par ouverture à l'instant présent, ainsi que la consultation d'une fiche complète avec ses avis.

La demande d'assistance couvre la déclaration de panne — type de véhicule, nature du problème présentée dans un ordre trié selon le véhicule choisi, niveau d'urgence, description libre, photographie facultative —, le choix d'un garage parmi les résultats classés, l'acceptation ou le refus par le garagiste, la mise en route, et le suivi bidirectionnel avec double estimation du temps d'arrivée.

La clôture couvre la reconnaissance mutuelle sur place, la double confirmation d'arrivée, la publication d'un avis rendue possible par la clôture, et le crédit des points de fidélité.

La fidélité couvre la consultation du solde de points et du grade, l'historique des mouvements, et le parrainage.

Le mode conduite couvre le démarrage, la pause et l'arrêt d'une session, l'affichage en direct de la vitesse, de la distance, de la durée et du nombre d'alertes, la production d'alertes simulées, et la consultation des statistiques passées.

2.5 Besoins non fonctionnels

Les besoins non fonctionnels sont ici moins décoratifs qu'à l'ordinaire, car plusieurs d'entre eux ont directement déterminé l'architecture.

Tableau 2 — Besoins non fonctionnels et exigences chiffrées

Domaine	Exigence	Justification
Performance de la recherche	La requête de proximité doit consommer un index géospatial	C'est la requête centrale, appelée à chaque ouverture de carte et à

	et non parcourir la table	chaque changement de tri
Fluidité cartographique	60 images par seconde au déplacement et au zoom, avec 20 marqueurs, sur Android milieu de gamme	Une carte saccadée est perçue comme une application cassée
Consommation de données	Environ 120 octets par position émise, soit de l'ordre de 36 Ko pour 20 minutes d'intervention	Les forfaits se comptent ; une application coûteuse en données est désinstallée
Tolérance au réseau	Le suivi doit survivre à une coupure et se remettre à jour seul au retour du réseau	La coupure n'est pas un incident, c'est le régime normal
Précision affichée	La précision réelle issue du capteur doit être montrée, jamais une valeur rassurante	Un badge qui ment détruit la confiance dans tout le reste
Lisibilité	Interface lisible en plein soleil, cible tactile d'au moins 44 par 44 pixels	L'utilisateur est dehors, debout, souvent sous stress
Sécurité des accès	Jeton d'accès court, jeton de rafraîchissement à rotation, jamais stocké en clair	Une fuite de la base ne doit pas donner de sessions utilisables
Intégrité de la fidélité	Aucun point crédité sans double confirmation ; aucun crédit rejouable	Les points sont une valeur monétaire
Internationalisation	Français par défaut, anglais prévu, libellés métier partagés serveur et mobile	Deux listes séparées finissent toujours par se contredire

Deux de ces exigences méritent un commentaire, car elles sont plus contraignantes qu'il n'y paraît.

La tolérance au réseau, d'abord. Il ne suffit pas de gérer l'erreur : il faut concevoir un mode de fonctionnement dégradé qui rende encore un service exact. La différence est importante. Gérer l'erreur, c'est afficher « connexion perdue » et attendre. Concevoir un mode dégradé, c'est basculer sur une autre voie d'acheminement, continuer d'afficher une donnée juste quoique moins fraîche, l'annoncer honnêtement à l'utilisateur, et se remettre à jour tout seul quand la connexion revient. La seconde attitude coûte plus cher en conception, et c'est celle qui a été retenue.

L'intégrité de la fidélité, ensuite. Elle impose une contrainte que la plupart des applications n'ont pas : la règle métier ne peut pas vivre uniquement dans le code applicatif. Un contrôle écrit dans une fonction est vrai tant que tous les chemins d'appel passent par cette fonction, ce qui cesse d'être vrai au troisième point d'entrée ajouté six mois plus tard. Une contrainte inscrite dans le schéma de la base reste vraie même quand le code se trompe. Le chapitre 5 détaille les cinq contraintes qui portent cette exigence.

2.6 Les acteurs

Le système reconnaît trois acteurs humains et un ensemble de services externes.

Le client est un automobiliste, propriétaire ou conducteur d'un véhicule. C'est l'acteur principal : il déclare les pannes, choisit les garages, suit les interventions, publie des avis et accumule des points. Il utilise l'application dans une situation qui n'est presque jamais confortable, et cette circonstance a des conséquences sur l'interface bien plus que sur le modèle de données.

Le garagiste est le propriétaire d'un atelier. Il s'inscrit depuis l'application, ce qui crée son garage et promeut son compte. Il reçoit les demandes qui lui sont adressées, les accepte ou les refuse, se déclare en route, et confirme son arrivée. Un même compte ne peut détenir qu'un seul garage, contrainte garantie par un index d'unicité et non par une vérification applicative, pour les raisons exposées au chapitre 5.

L'administrateur vérifie les dossiers de garage déposés depuis l'application et prononce éventuellement leur certification. Le document distingue soigneusement ces deux actes : vérifier, c'est constater que le garage existe ; certifier, c'est répondre de sa qualité. Un garage vérifié mais non certifié est le cas normal, et un garage non vérifié ne peut pas être actif, ce qui l'empêche structurellement d'apparaître dans les résultats de recherche.

Les services externes sont le fournisseur de tuiles cartographiques, le service de stockage des photographies, et le moteur de calcul d'itinéraire. Ils n'ont pas d'initiative propre et n'apparaissent donc pas comme acteurs dans le diagramme de cas d'utilisation, mais ils conditionnent le fonctionnement et sont traités au chapitre 6.

La solution proposée

3.1 Principe

GeoCras est une application mobile unique, servant deux rôles distincts derrière le même compte, adossée à une interface de programmation qui détient la donnée métier.

Le principe tient en une phrase : la position remplace la description. Tout ce que l'application fait de particulier découle de cette substitution. Le conducteur n'explique plus où il est, son téléphone le dit. Le garagiste ne reconstitue plus un itinéraire à partir de repères, il le voit tracé. Le classement des ateliers ne se fait plus de mémoire, il se calcule sur une distance réelle, une note pondérée et une certification vérifiée. Et l'arrivée n'est plus une affirmation, c'est un événement horodaté que les deux parties confirment.

Le service se déroule en cinq temps. La position est captée au lancement, sans action de l'utilisateur. Le bouton SOS ouvre une déclaration de panne courte, où la liste des problèmes proposés est triée selon le véhicule déclaré, la panne la plus probable en tête. L'envoi de cette déclaration retourne, dans le même aller-retour réseau, la liste des garages classés par pertinence, affichés sur une carte avec des marqueurs numérotés. Le client en choisit un ; le garagiste reçoit la demande, l'accepte ou la refuse, puis se déclare en route. À partir de là, les deux parties se voient avancer l'une vers l'autre, avec deux temps d'arrivée estimés recalculés en continu par le serveur. Enfin, quand la distance qui les sépare devient une distance de vue, l'application leur demande à chacune de confirmer qu'elles se sont trouvées, et c'est cette double confirmation qui clôt l'intervention et déclenche le crédit de points.

3.2 Ce qui distingue la solution

Quatre partis pris méritent d'être isolés, parce qu'ils ne vont pas de soi et qu'ils ont chacun un coût.

Le premier est que le classement est calculé par le serveur et jamais recalculé par le mobile. Un rang affiché à l'écran est un rang produit par la requête SQL, transporté tel quel jusqu'au marqueur. Cela paraît anodin ; c'est en réalité ce qui garantit que le numéro 2 sur la carte est le même que le numéro 2 dans la liste, y compris après un changement de tri, une reconnexion ou un rafraîchissement partiel. Un classement recalculé localement diverge du serveur dès que les données bougent, et cette divergence est invisible en développement puis embarrassante en production.

Le deuxième est que l'itinéraire est tracé dans l'application, et non délégué par un lien vers une carte tierce. Le lien externe aurait été bien plus simple à écrire. Il est incompatible avec le produit : il fait sortir l'utilisateur de l'application, donc interrompt le suivi, donc supprime la

double confirmation, donc détruit la preuve sur laquelle repose toute la fidélité. Un choix d'apparence pratique aurait fait tomber la fonction centrale.

Le troisième est que le temps d'arrivée estimé est calculé par le serveur pour les deux parties. Il aurait été plus économique de laisser chaque téléphone calculer le sien. Deux raisons l'interdisent : les deux parties doivent voir le même chiffre, faute de quoi elles se contredisent au téléphone ; et cet estimé alimente la détection de fraude, ce qui suppose qu'il ne soit pas produit par un appareil que l'on cherche précisément à surveiller.

Le quatrième est que les règles métier sensibles sont inscrites dans le schéma de la base et non dans le code applicatif. C'est plus rigide, cela oblige à écrire des migrations, et cela rend certaines évolutions plus lourdes. C'est aussi la seule manière d'obtenir une garantie qui survive aux six prochains mois de développement.

3.3 Périmètre de la première version

Il est utile d'énoncer ce que la première version ne fait pas, et pourquoi, car un périmètre non écrit se remplit tout seul.

Le mode conduite est une simulation. Aucun capteur, aucune caméra, aucun radar. Les alertes sont produites par un moteur d'événements dont la crédibilité est travaillée — profil de vitesse plausible, alertes cohérentes avec la vitesse courante — mais qui ne mesure rien du monde extérieur. Ce point est assumé et affiché : il serait malhonnête de laisser croire à un conducteur qu'une alerte d'angle mort correspond à une observation.

Le paiement en ligne n'est pas intégré. La conversion des points en Mobile Money est décrite dans le barème mais n'est jamais automatique dans cette version : elle passe par une revue. La raison est exposée au chapitre 5 et tient entièrement au risque de fraude.

Le moteur de calcul d'itinéraire n'est pas encore auto-hébergé. La cible est un serveur de routage sur l'extrait cartographique du Cameroun ; en attendant, la durée de trajet est approchée par un calcul géométrique et l'interface la présente explicitement comme provisoire, plutôt que d'afficher un chiffre précis qui serait faux.

Enfin, l'anglais est prévu dans toute la chaîne — les libellés métier portent déjà leurs deux versions — mais l'interface n'est livrée qu'en français.

3.4 Les trois risques identifiés

Le projet a été planifié autour de trois risques, classés par gravité, et l'ordre de construction en découle directement.

Le premier et le plus grave est la chaîne de confiance de la fidélité. Les points ont vocation à devenir de l'argent. La double confirmation d'arrivée prouve que deux personnes se sont mises d'accord, elle ne prouve pas qu'une intervention a eu lieu. Deux comptes complices qui se confirment mutuellement suffisent à faire imprimer de la valeur au système. Ce risque est

traité au chapitre 5 par un ensemble de garde-fous dont le plus important est la preuve de mouvement.

Le deuxième est la qualité du signal de localisation et du réseau. Vingt à cinquante mètres de précision en centre-ville, une couverture qui oscille, des coupures franches. Les symptômes sont connus : des temps d'arrivée qui sautent, des positions qui se téléportent, un indicateur de précision qui ment. Le traitement combine un filtrage des mesures, le rejet des déplacements physiquement implausibles, l'affichage de la précision réelle et un mode dégradé conçu dès le départ.

Le troisième est technique et concerne la couche cartographique. La bibliothèque de rendu retenue doit fonctionner sur la nouvelle architecture de la plateforme mobile, ce qui n'est pas confirmé pour la version du kit de développement employée. C'est le risque le plus susceptible de coûter une semaine entière, et il conditionne tous les écrans qui affichent une carte. Il a donc été placé en tout premier dans l'ordre de construction, avant le moindre écran d'habillage, avec deux issues de secours documentées à l'avance.

Spécifications générales

4.1 Architecture générale du système

Le système se compose de quatre couches, dont une, le paquet de contrats partagés, n'existe que pour empêcher les deux extrémités de diverger.

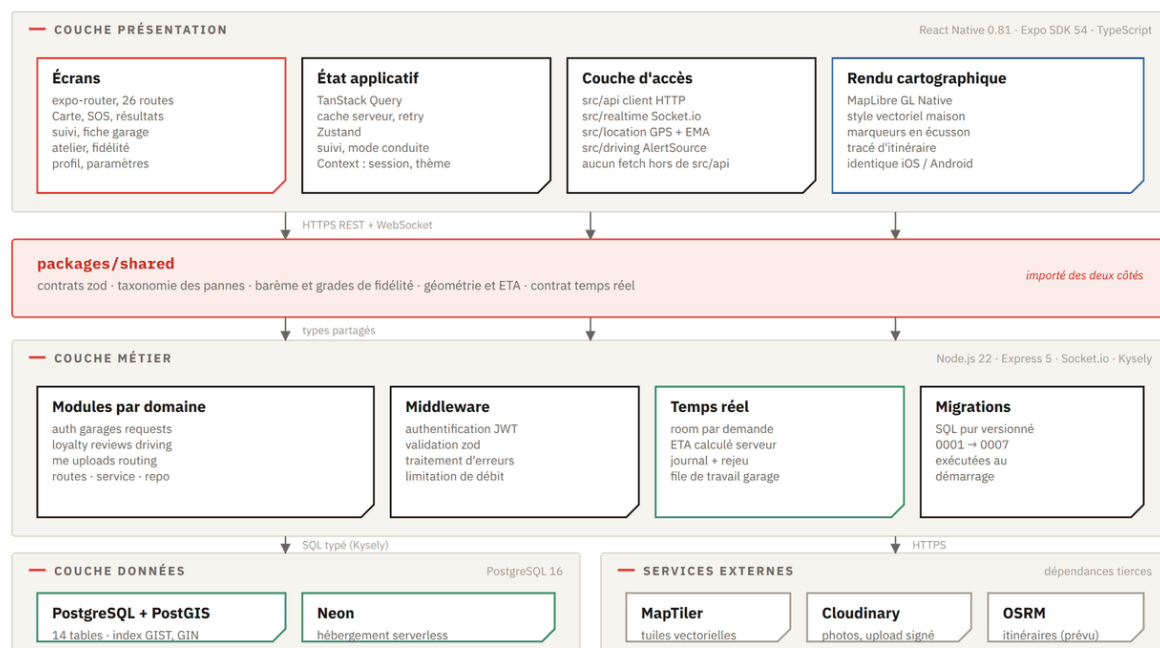


Figure 1 — Architecture générale : couches, responsabilités et dépendances externes

La couche de présentation est une application React Native construite avec Expo, dont le routage repose sur l'arborescence de fichiers. Elle contient les écrans, l'état applicatif, la couche d'accès au réseau et le rendu cartographique. Une règle y est appliquée strictement : aucun appel réseau n'est écrit dans un composant, tout passe par le dossier dédié. Cette règle a un effet concret sur le mode dégradé, qui n'aurait pas été implémentable si les appels avaient été dispersés.

Le paquet de contrats partagés occupe une position particulière, entre le mobile et le serveur, importé par les deux. Il porte les schémas de validation, qui sont la source unique des types d'interface ; la taxonomie des pannes, dont l'ordre d'affichage constitue le tri automatique visible à l'écran ; le barème et les grades de fidélité ; les fonctions géométriques ; et le contrat du canal temps réel, y compris les noms d'événements. L'intérêt est qu'une divergence entre les deux extrémités ne produit pas un bogue à l'exécution mais une erreur de compilation.

La couche métier est un serveur Express découpé par domaine et non par couche technique. Chaque domaine réunit ses routes, son service et son accès aux données dans un même dossier. Ce découpage a un effet mesurable sur le coût des évolutions : ajouter un niveau

d'urgence à la demande d'assistance se fait dans un seul dossier, là où un découpage horizontal aurait imposé de toucher quatre répertoires pour un seul champ.

La couche de données est un PostgreSQL enrichi de son extension géospatiale, hébergé sur une offre serverless. Les migrations sont des fichiers SQL écrits à la main, versionnés et exécutés dans l'ordre.

4.2 Acteurs et droits

Tableau 3 — Acteurs du système et opérations autorisées

Acteur	Opérations autorisées	Restrictions
Visiteur non authentifié	Rechercher les garages à proximité, consulter une fiche et ses avis	Aucune demande d'assistance, aucun avis
Client	Toutes les opérations du visiteur, plus déclarer une panne, choisir un garage, suivre, confirmer son arrivée, annuler, publier un avis, gérer ses véhicules, consulter sa fidélité, utiliser le mode conduite	Une seule demande active à la fois ; un avis par demande clôturée
Garagiste	Toutes les opérations du client, plus recevoir les demandes qui lui sont adressées, accepter, refuser, se déclarer en route, confirmer son arrivée, gérer la fiche de son garage	Un seul garage par compte ; garage inactif tant qu'il n'est pas vérifié
Administrateur	Vérifier un dossier de garage, prononcer la certification	La certification est distincte de la vérification

Le fait que la recherche de garages soit ouverte sans authentification est un choix délibéré. Obliger quelqu'un à créer un compte avant de lui montrer les garages autour de lui reviendrait à lui demander de taper un mot de passe pendant que son véhicule est en panne. L'authentification n'est exigée qu'au moment où la demande d'assistance est réellement émise, c'est-à-dire au moment où l'on a besoin de savoir à qui l'on parle.

4.3 Diagramme de cas d'utilisation

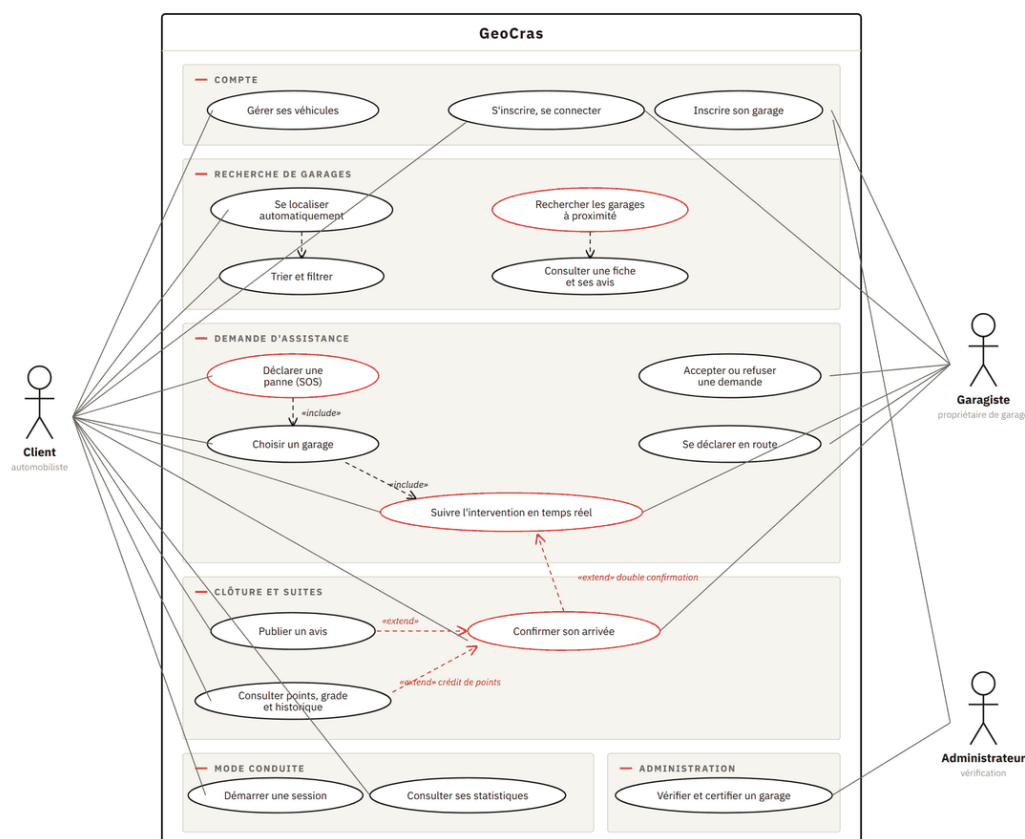


Figure 2 — Diagramme de cas d'utilisation : acteurs, paquets fonctionnels et relations d'inclusion et d'extension

Le diagramme fait apparaître trois structures qu'il vaut la peine de commenter.

Les relations d'inclusion traduisent des dépendances obligatoires. Déclarer une panne inclut se localiser automatiquement : il n'existe pas de chemin où l'on ouvre le formulaire de panne sans que l'acquisition de position ait été tentée. De même, choisir un garage inclut le suivi de l'intervention, parce que le choix bascule immédiatement l'écran vers l'attente puis vers le suivi, sans étape intermédiaire où l'utilisateur devrait décider d'ouvrir un suivi.

Les relations d'extension traduisent des comportements conditionnels. Publier un avis étend la confirmation d'arrivée : l'avis n'est possible que si l'intervention a été clôturée, et la clôture n'est possible que si les deux parties ont confirmé. La consultation des points étend également la confirmation, parce que c'est elle qui déclenche le crédit.

Enfin, la confirmation d'arrivée est le seul cas d'utilisation associé aux deux acteurs opérationnels. Cette symétrie n'est pas cosmétique : elle est la traduction, au niveau des cas d'utilisation, de la règle qui fonde toute la chaîne de confiance.

4.4 Description des cas d'utilisation principaux

4.4.1 Déclarer une panne

L'acteur principal est le client. La précondition est qu'il soit authentifié et qu'il n'ait pas déjà une demande en cours. Le déclenchement est l'appui sur le bouton SOS.

Le déroulement nominal enchaîne l'acquisition de la position, le choix du type de véhicule parmi voiture, moto, camion et une catégorie ouverte, l'affichage des pannes correspondantes triées par probabilité décroissante, le choix de la panne, le renseignement du niveau d'urgence, une description libre, une photographie facultative, puis l'envoi. Le système valide la demande, l'enregistre, la journalise et retourne dans la même réponse la liste des garages classés.

Deux variantes méritent d'être notées. Si la position n'a pas pu être obtenue, l'application affiche un bandeau de reprise de la localisation et laisse néanmoins l'utilisateur poursuivre : rester bloqué priverait de l'application quelqu'un qui a refusé la permission de localisation. Si le type de véhicule ou la nature de la panne relève de la catégorie ouverte, un libellé libre devient obligatoire — règle appliquée simultanément par le mobile, qui désactive le bouton de validation, et par le serveur, qui refuse la demande, les deux s'appuyant sur la même définition partagée.

La postcondition est une demande en attente de choix de garage, associée à une liste ordonnée de candidats.

4.4.2 Suivre l'intervention

Les acteurs sont le client et le garagiste. La précondition est une demande acceptée dont le garagiste s'est déclaré en route.

Le déroulement est continu plutôt que séquentiel. Chaque partie émet sa position selon une double condition : au plus une émission toutes les quatre secondes, et seulement si elle s'est déplacée d'au moins quinze mètres. Le serveur enregistre chaque position, calcule les deux temps d'arrivée estimés et les diffuse aux deux parties avec l'horodatage d'émission. L'interface affiche l'âge réel de la donnée, calculé par différence avec cet horodatage, plutôt qu'un intervalle nominal qui serait faux dès la première latence.

La variante importante est la perte de connexion. Le canal temps réel est alors déclaré mort, l'application bascule sur une interrogation périodique de la route d'état, et affiche un bandeau de connexion dégradée. Au retour du réseau, le client se réannonce en indiquant le dernier numéro de séquence qu'il a reçu, et le serveur lui rejoue les événements manquants. Ce mécanisme est détaillé au chapitre 5.

4.4.3 Confirmer l'arrivée

Les acteurs sont le client et le garagiste, chacun de son côté. La précondition est une demande en cours de suivi.

Lorsque la distance entre les deux parties passe sous un seuil de cent vingt mètres, l'application propose à chacune de confirmer qu'elle voit l'autre. Ce seuil peut surprendre : on imaginerait spontanément dix mètres. Trois raisons l'écartent. La précision d'un téléphone en ville se situe entre dix et trente mètres, et davantage entre deux immeubles, si bien qu'un seuil de dix mètres vivrait entièrement dans le bruit de mesure — il ne se déclencherait jamais, ou il clignoterait. La position transmise a par ailleurs de l'âge, puisqu'on n'émet qu'au-delà de quinze mètres parcourus et toutes les quatre secondes : à quarante kilomètres par heure, le dernier point connu date déjà d'une quarantaine de mètres. Enfin, cent vingt mètres correspond à une portée de vue réelle : sur une rue de Yaoundé, un véhicule arrêté à cette distance se repère.

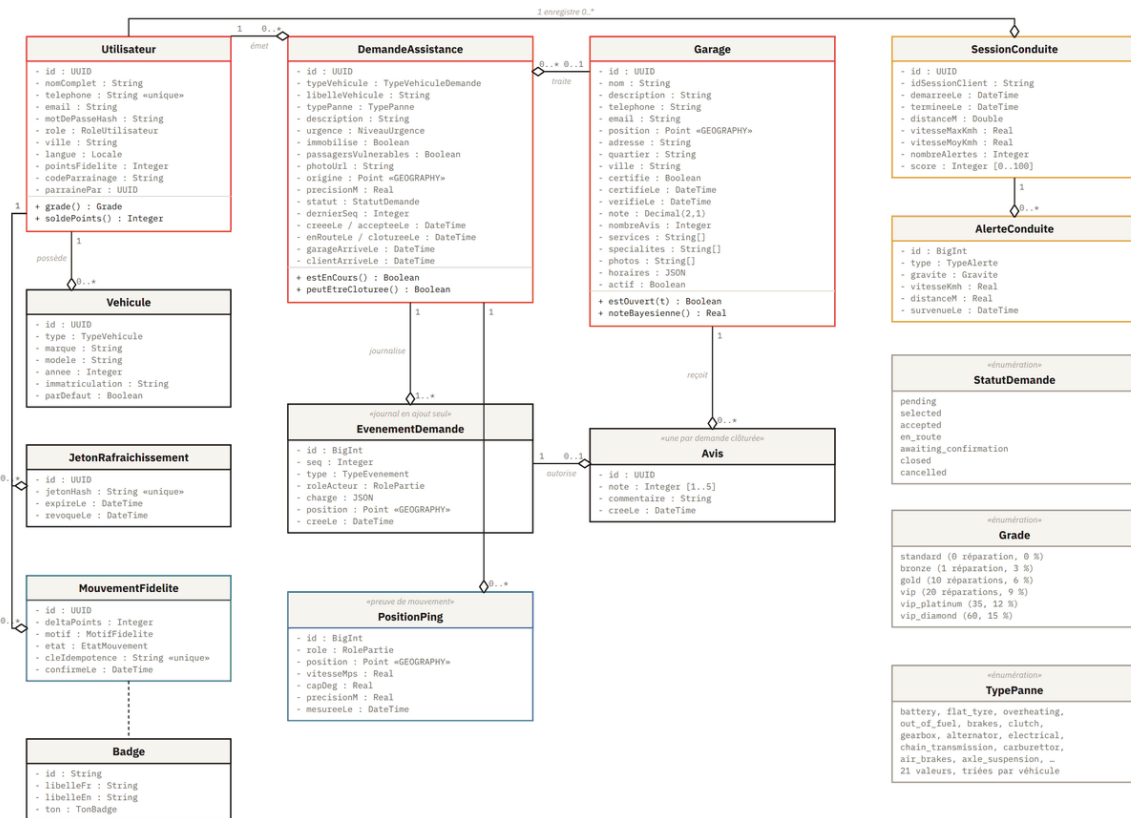
La première confirmation fait passer la demande dans un état d'attente de la seconde. La seconde clôt la demande. L'opération est idempotente : une confirmation rejouée, par exemple à cause d'un renvoi réseau, ne produit aucun effet supplémentaire.

La postcondition est une demande clôturée, un avis rendu possible, et un mouvement de fidélité créé à l'état différé.

Spécifications détaillées

5.1 Diagramme de classes

Le modèle du domaine comporte douze classes persistantes et un ensemble d'énumérations qui vivent dans le paquet partagé. Le diagramme ci-dessous en donne la structure complète, attributs et cardinalités comprises.



Composition (losange plein côté contenu) : le journal, les pings et les alertes n'existent pas sans leur parent.

Figure 3 — Diagramme de classes du domaine : entités, attributs, opérations et cardinalités

Trois compositions structurent le modèle, et il faut noter qu'il s'agit bien de compositions et non de simples agrégations : les entités portées n'ont aucune existence indépendante de leur conteneur. Le journal d'événements n'existe pas sans sa demande. Les relevés de position n'existent pas sans elle non plus. Les alertes de conduite n'existent pas sans leur session. La suppression du conteneur emporte celle des éléments portés, ce qui se traduit en base par des suppressions en cascade.

Deux points du modèle demandent une explication, parce qu'ils s'écartent de ce que l'on écrirait spontanément.

Le premier concerne la classe Utilisateur et son attribut de points de fidélité. Cet attribut est un cache, pas une vérité. La vérité comptable est la somme des mouvements du journal de

fidélité ; la colonne n'existe que pour éviter une agrégation à chaque affichage d'écran. Le distinguer explicitement évite qu'un développeur futur ne la mette à jour directement, ce qui reviendrait à créditer des points sans laisser de trace.

Le second concerne la séparation entre les points et les grades. Ce sont deux mesures distinctes qui ne disent pas la même chose. Les points sont une monnaie : ils s'accumulent, se dépensent, et peuvent provenir d'un avis publié ou d'un parrainage sans qu'aucun véhicule n'ait été réparé. Le grade est une ancienneté de client : il ne se dépense pas, et il ne récompense que ce qui s'est réellement passé au bord de la route, c'est-à-dire un compte de réparations terminées. Les faire dépendre l'un de l'autre reviendrait à rétrograder quelqu'un qui convertit ses points en remise, ce qui serait absurde.

Tableau 4 — Grades de fidélité et remises associées chez les garages certifiés

Grade	Réparations terminées	Remise
Membre	0	0 %
Membre Bronze	1	3 %
Membre Or	10	6 %
Membre VIP	20	9 %
VIP Platine	35	12 %
VIP Diamant	60	15 %

Tableau 5 — Barème de points par action vérifiée

Action	Points	Bénéficiaire
Intervention terminée	50	Le client
Avis publié	20	L'auteur de l'avis
Parrainage abouti	100	Le parrain, à la première intervention du filleul
Inscription parrainée	30	Le filleul, à la même occasion

5.2 Diagramme d'états-transitions de la demande d'assistance

La demande d'assistance est l'entité qui porte le cycle de vie du produit. Sa machine à états compte sept états dont deux terminaux.

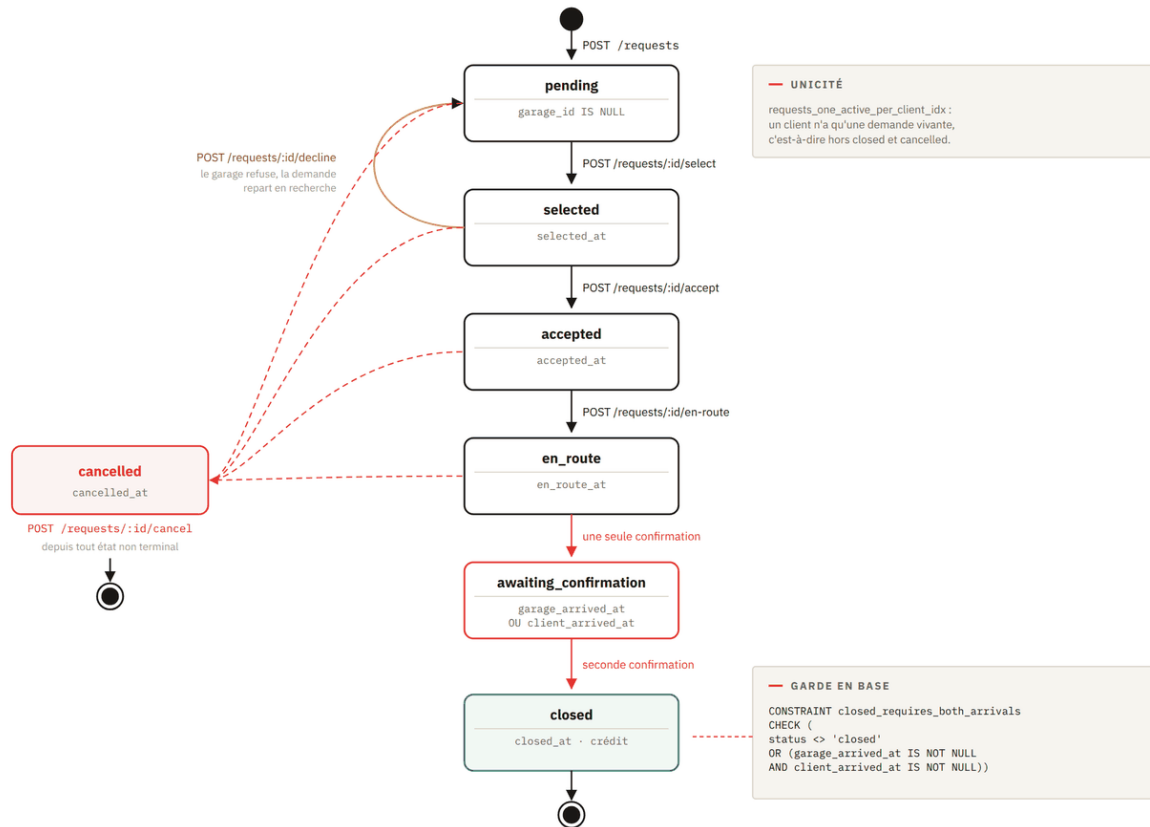


Figure 4 — Cycle de vie d'une demande d'assistance, avec la garde inscrite en base

Le chemin nominal est linéaire : la demande naît en attente de choix de garage, passe en attente de réponse dès qu'un garage est retenu, puis en acceptée, puis en route lorsque le garagiste se déclare parti. Elle entre ensuite dans un état intermédiaire d'attente de la seconde confirmation, franchi lorsque la première des deux parties confirme son arrivée, et se clôt à la seconde confirmation.

Deux transitions sortent de ce chemin et méritent d'être commentées.

La première est le refus. Un garagiste qui ne peut pas intervenir n'avait initialement qu'une seule sortie : annuler. C'est-à-dire fermer la demande de quelqu'un qui est en panne au bord d'une route, et l'obliger à tout ressaisir pour en trouver un autre. Le refus a donc été introduit comme une transition distincte, qui est la seule marche arrière de la machine : elle ramène la demande à son état initial en lui retirant uniquement son garage. La demande conserve son identifiant, son journal et son ancienneté. Cette distinction n'est pas cosmétique : elle préserve dans le journal la seule trace qui permette de mesurer le taux de refus d'un garage, et qui explique après coup pourquoi une demande a traversé trois ateliers avant d'aboutir.

La seconde est l'annulation, accessible depuis tout état non terminal et réservée au client. Elle est terminale.

La garde la plus importante de cette machine n'est pas écrite dans le code du serveur mais dans le schéma de la base :

La contrainte qui rend structurellement impossible une clôture non prouvée

```
CONSTRAINT closed_requires_both_arrivals CHECK (
  status <> 'closed'
  OR (garage_arrived_at IS NOT NULL AND client_arrived_at IS NOT NULL)
)
```

Aucun chemin de code, présent ou futur, ne peut contourner cette contrainte. C'est ce qui garantit qu'aucun point ne sera jamais crédité sans les deux confirmations, quelle que soit la manière dont le serveur évoluera.

Pourquoi une contrainte plutôt qu'une vérification

Une vérification écrite dans un service est vraie tant que tous les appelants passent par ce service. Elle cesse de l'être au troisième point d'entrée ajouté six mois plus tard par quelqu'un qui ignore la règle. Une contrainte inscrite dans le schéma reste vraie même quand le code se trompe. Le coût est une migration à écrire ; le bénéfice est une garantie qui ne se dégrade pas avec le temps.

5.3 Diagramme de séquence du parcours d'assistance

Le diagramme suivant déroule un parcours complet, de l'appui sur le bouton d'urgence jusqu'au crédit de fidélité, en faisant apparaître les six participants réels.

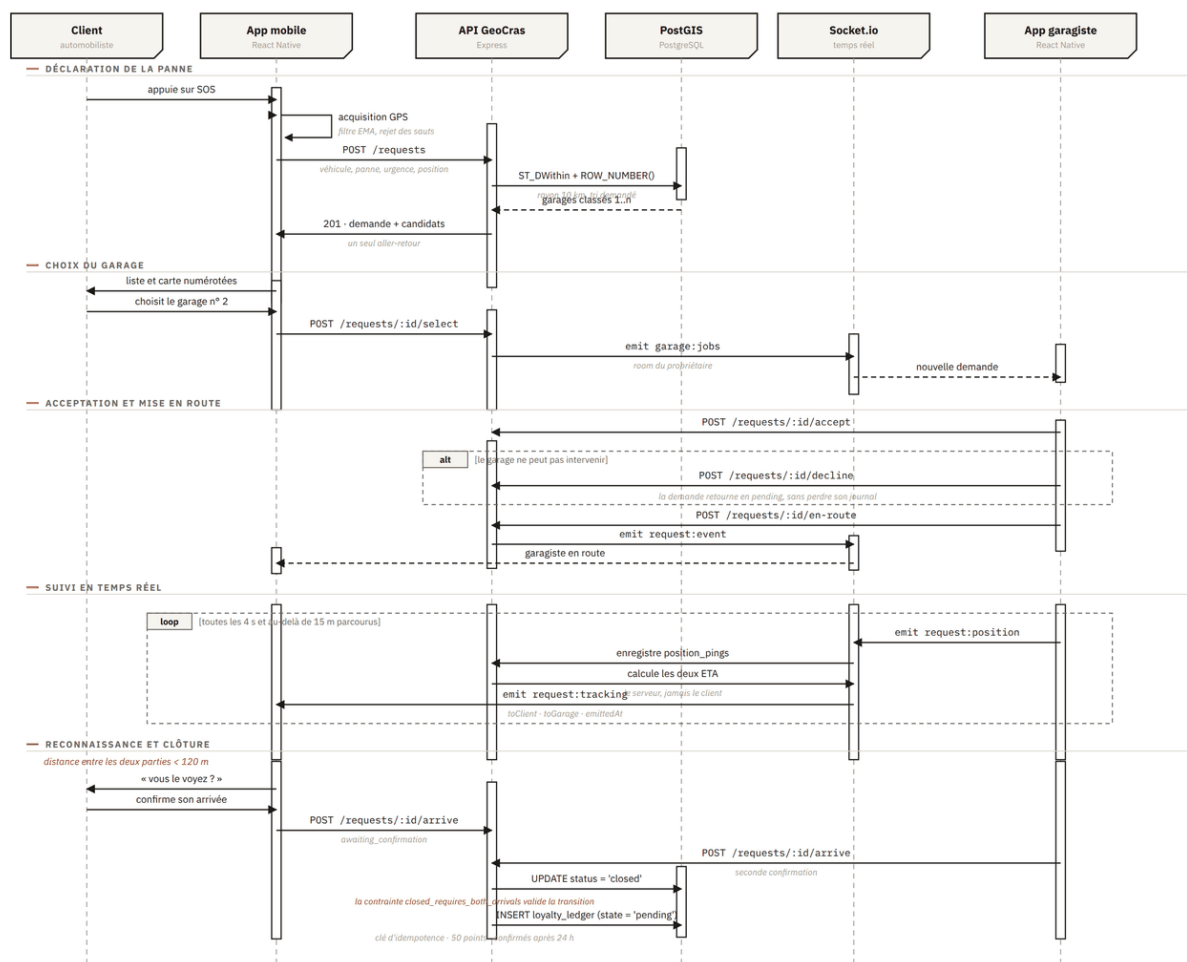


Figure 5 — Séquence complète d'une intervention, de la déclaration de panne à la clôture

Plusieurs points de ce déroulé sont des décisions de conception plutôt que des évidences.

L'envoi de la déclaration de panne retourne les garages classés dans la même réponse. Il aurait été plus régulier de créer la demande puis d'interroger une seconde route pour obtenir les candidats. Sur un réseau qui met deux secondes à répondre, cette régularité coûte deux secondes à quelqu'un qui est arrêté sur une chaussée. Le choix a été d'économiser l'aller-retour.

La notification du garagiste ne passe pas par la salle de la demande, contrairement à tous les autres messages. Elle ne le pourrait pas : au moment où un appel lui est adressé, le garagiste ne connaît pas encore cette demande et n'a donc rejoint aucune salle. Il est abonné à sa propre salle utilisateur dès l'ouverture de sa connexion, et c'est là que l'appel arrive. Le serveur lui pousse d'ailleurs la liste entière de sa file de travail plutôt qu'un signal l'invitant à recharger : la liste tient en quelques centaines d'octets, là où un aller-retour supplémentaire sur un réseau lent repousse d'autant le moment où il voit la demande.

Le calcul des deux temps d'arrivée est effectué par le serveur à chaque relevé de position reçu, puis diffusé aux deux parties avec l'horodatage d'émission. C'est ce qui permet à l'interface d'afficher l'âge réel de la donnée.

Enfin, la clôture et le crédit sont deux opérations distinctes exécutées dans la même transaction. Le crédit est créé à l'état différé, avec une clé d'idempotence qui interdit à une requête rejouée de créditer deux fois.

5.4 Diagramme de séquence du mode dégradé

Le comportement en réseau dégradé fait l'objet d'un diagramme séparé, parce qu'il constitue une exigence à part entière et non une gestion d'erreur.

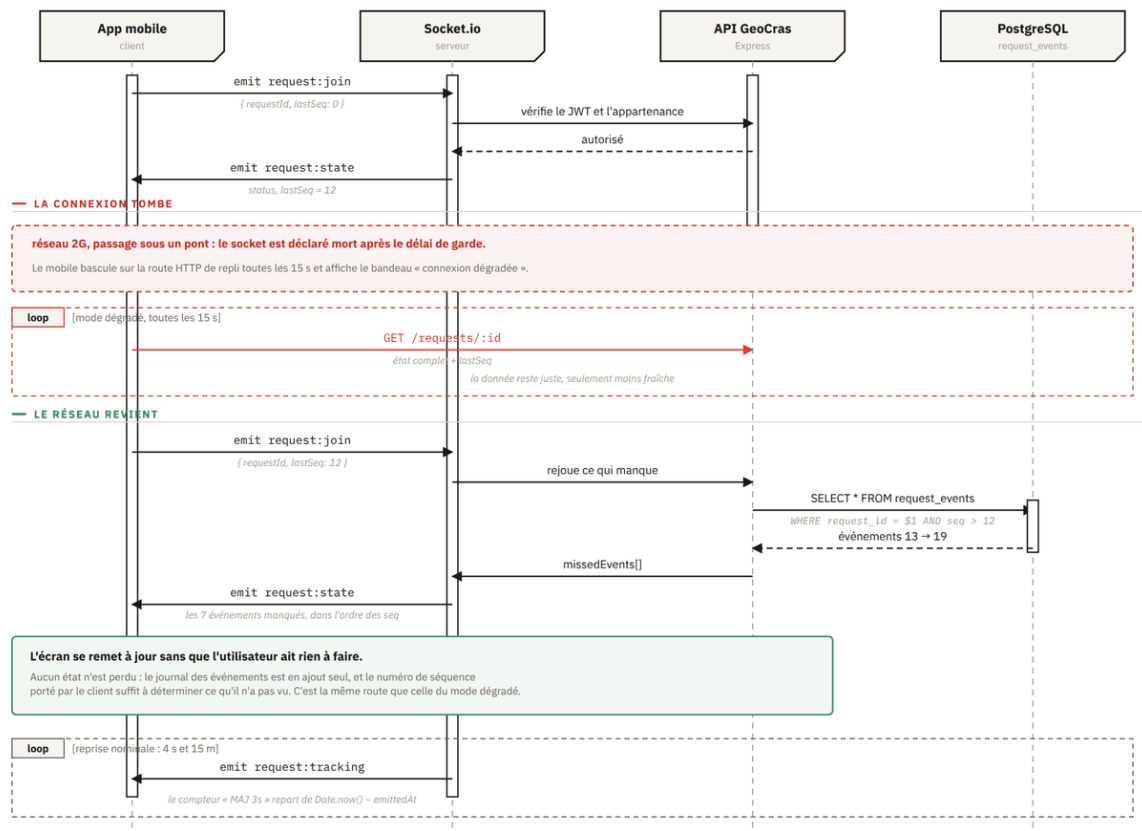


Figure 6 — Perte de connexion, bascule en mode dégradé et rattrapage par rejeu du journal

Le mécanisme repose entièrement sur le fait que le journal des événements est en ajout seul et numéroté. Chaque événement porte un numéro de séquence unique au sein de sa demande, et le client conserve le dernier numéro qu'il a reçu. Quand le canal temps réel meurt, l'application bascule sur une interrogation périodique de la route d'état — la même route, exactement, qui sert au premier chargement — et affiche un bandeau qui dit honnêtement que la connexion est dégradée. La donnée affichée reste juste, elle est simplement moins fraîche, et l'indicateur d'âge le montre.

Au retour du réseau, le client se réannonce en transmettant son dernier numéro de séquence. Le serveur lit dans le journal tous les événements postérieurs et les lui rejoue dans l'ordre. L'écran se remet à jour sans que l'utilisateur ait rien à faire, et sans qu'aucun état n'ait été perdu.

Ce dispositif a un effet secondaire appréciable : il rend le mode dégradé et le premier chargement structurellement identiques. Il n'existe donc pas de chemin de code rarement emprunté, donc rarement testé, qui ne fonctionnerait que le jour de la coupure.

5.5 Diagramme d'activité de la déclaration de panne

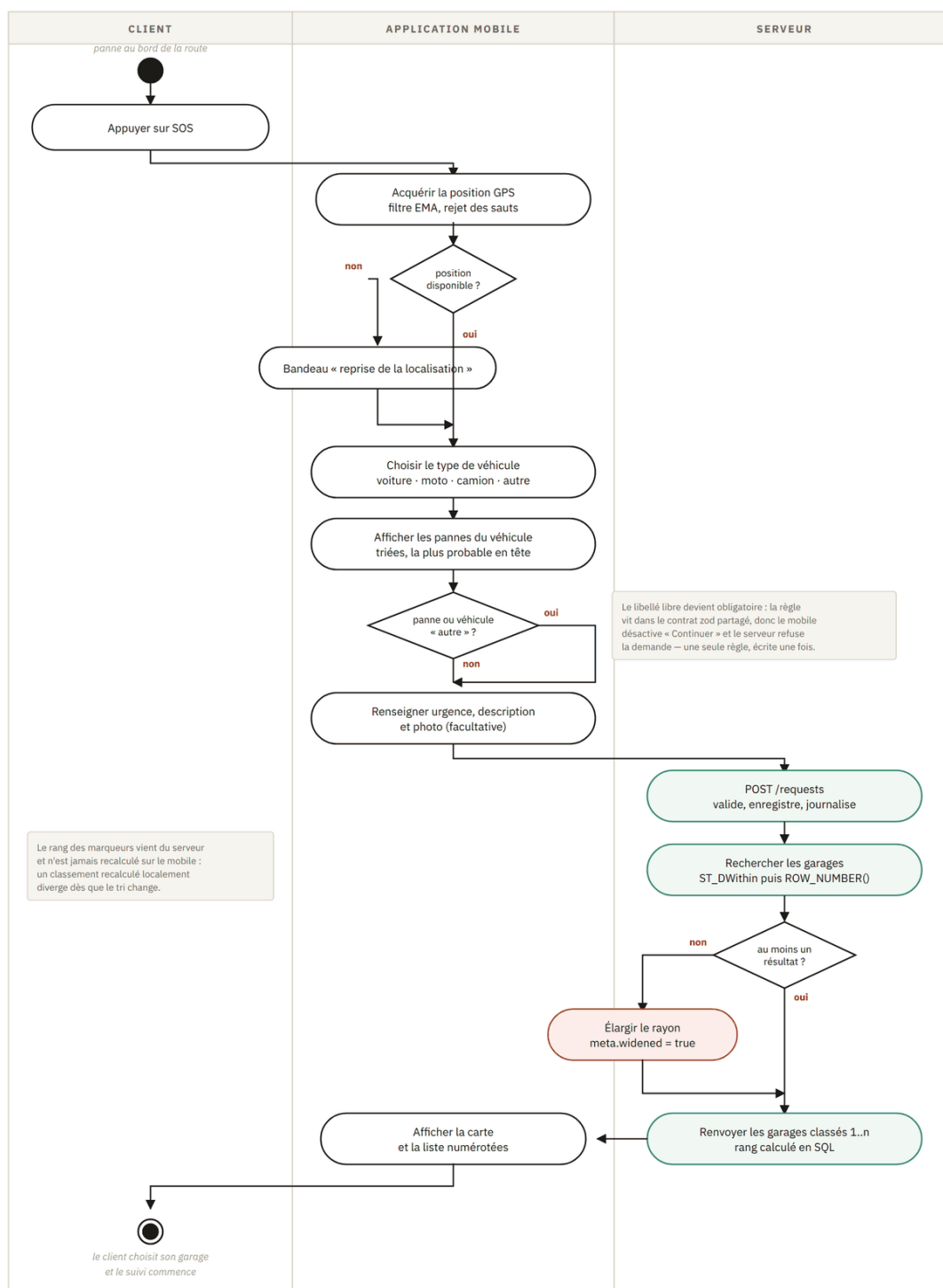


Figure 7 — Activité de déclaration de panne, réparties entre le client, l'application et le serveur

Le diagramme fait apparaître deux points de décision dont le traitement est plus subtil qu'il n'y paraît.

Le premier est l'échec d'acquisition de la position. L'application ne bloque pas : elle affiche un bandeau de reprise et laisse le parcours se poursuivre. C'est une règle générale du produit —

une permission refusée ou un capteur indisponible sont des issues légitimes, pas des impasses.

Le second est l'absence de résultat dans le rayon de recherche. Plutôt que de retourner une liste vide, le serveur élargit la recherche et remonte le garage le plus proche en signalant explicitement que le rayon a été élargi. Le raisonnement est de bon sens : quelqu'un en panne à qui l'on affiche une liste vide n'a rien à faire de la rigueur du rayon paramétré.

5.6 Schéma de la base de données

La base compte quatorze tables. Le schéma est décrit ci-dessous avec ses clés, ses contraintes et ses index décisifs.

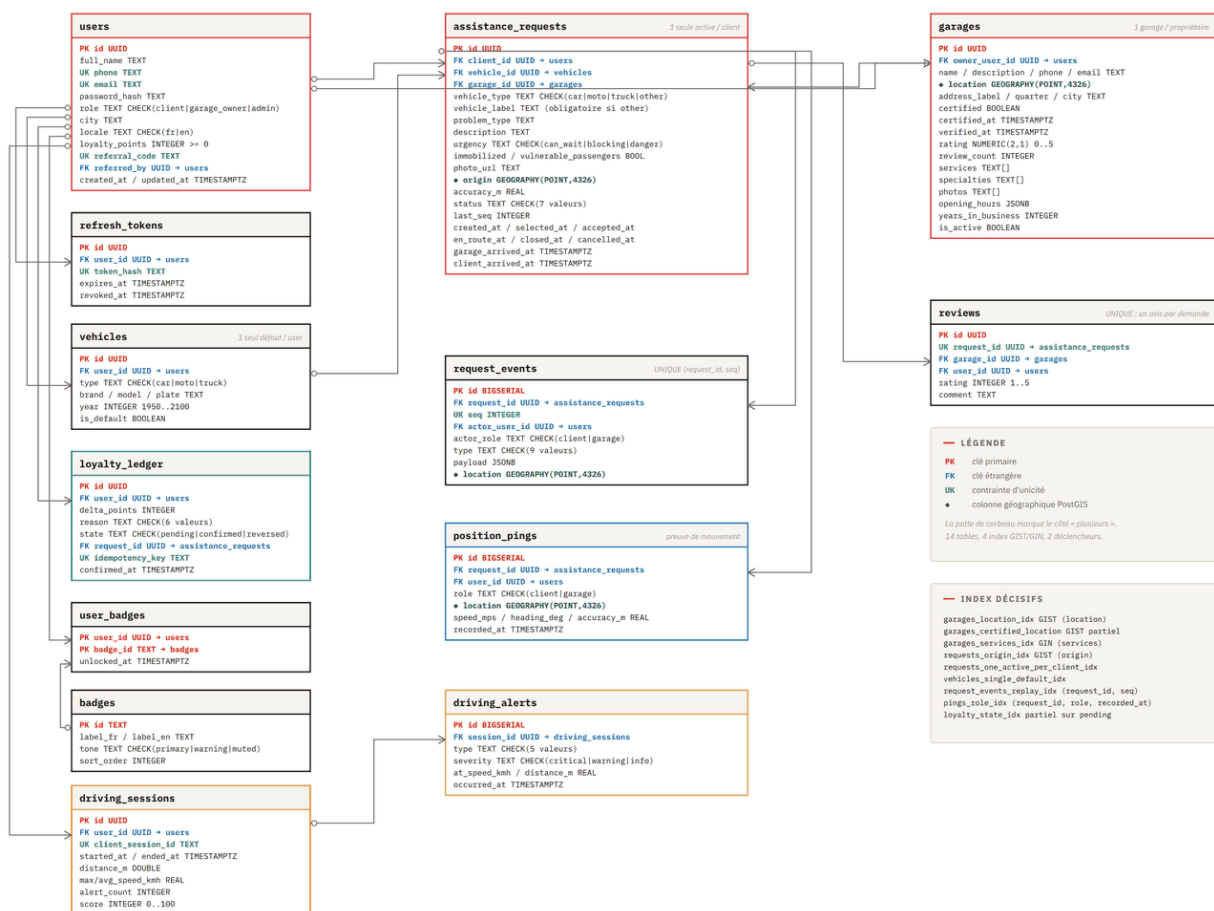


Figure 8 — Modèle physique de données : quatorze tables, clés, contraintes et index

Deux partis pris traversent tout le schéma.

Le premier est l'usage de colonnes textuelles assorties de contraintes de vérification plutôt que de types énumérés natifs. Ajouter une valeur à un type énuméré est une opération verrouillante sur PostgreSQL ; modifier une contrainte de vérification ne l'est pas. Comme la taxonomie des pannes et les niveaux d'urgence sont appelés à évoluer, le choix a été fait de privilégier la souplesse d'évolution.

Le second est l'usage du type géographique plutôt que géométrique pour toutes les positions. Le type géographique rend directement des mètres en tenant compte de la courbure terrestre, sans conversion manuelle et sans choix de projection à justifier. Sur un produit dont la fonction centrale est une distance en mètres, c'est le type qui dit la vérité par défaut.

5.6.1 Les règles métier inscrites comme contraintes

Cinq contraintes portent des règles métier que l'on aurait pu écrire dans le code applicatif. Elles sont listées ici avec ce que chacune garantit.

Tableau 6 — Contraintes de base de données portant des règles métier

Contrainte	Ce qu'elle garantit
closed_requires_both_arrivals	Une demande ne peut pas être clôturée sans les deux confirmations d'arrivée, donc aucun point ne peut être crédité sans elles
reviews.request_id UNIQUE	Un avis suppose une intervention réellement clôturée, et un client qui revient trois fois note trois fois
loyalty_ledger.idempotency_key UNIQUE	Une requête rejouée ne peut jamais créditer deux fois
vehicles_single_default_idx	Un seul véhicule par défaut par utilisateur, y compris en écriture concurrente
requests_one_active_per_client_idx	Un client ne peut pas ouvrir dix demandes d'assistance en parallèle
garages_owner_idx	Un compte ne peut détenir qu'un seul garage
active_requires_verification	Un garage non vérifié ne peut pas être actif, donc ne peut pas apparaître dans les résultats
vehicle_other_requires_label	Un véhicule de catégorie ouverte exige un libellé exploitable par le garagiste

Deux de ces contraintes méritent un développement.

L'unicité du garage par propriétaire répond à une situation concrète. L'inscription en tant que garagiste crée un garage et promeut le compte. Le service vérifie déjà qu'il n'en existe pas d'autre, mais deux envois du formulaire à une seconde d'intervalle — réseau lent, l'utilisateur appuie deux fois — passeraient tous deux la vérification avant qu'aucun n'ait inséré. On obtiendrait deux garages au même endroit, dont un fantôme que son propriétaire ne verrait jamais dans son espace mais qui remonterait dans les résultats de recherche. L'index d'unicité rend cette situation impossible.

La liaison entre activité et vérification répond à un risque de sécurité. Tant que les garages entraient exclusivement par un jeu de données constitué à la main, ils étaient vérifiés par construction. L'inscription depuis l'application ouvre la porte à des dossiers déclarés par leur propre auteur, et un garage inventé qui remonte dans une recherche envoie quelqu'un en

panne vers une adresse qui n'existe pas. La contrainte garantit qu'aucun chemin de code, y compris une future route de modification mal gardée, ne peut faire entrer un dossier en attente dans les résultats.

5.6.2 La requête de pertinence

La recherche des garages à proximité est la requête centrale du produit. Elle est appelée à chaque ouverture de la carte et à chaque changement de tri. Deux invariants la gouvernent.

Le premier est que le filtre de distance s'applique avant le tri. C'est ce qui fait consommer l'index géospatial. Un tri par distance sans filtre préalable produit un résultat correct mais parcourt la table entière — ce qui ne se voit pas tant que la base est petite, et s'effondre passé quelques milliers de garages. Un test automatisé vérifie le plan d'exécution pour cette raison précise.

Le second est que le rang est calculé en SQL, par une fonction de fenêtrage, et transporté tel quel jusqu'à l'interface.

Expression de rang : un seul plan couvre les trois tris

```
ROW_NUMBER() OVER (ORDER BY
  CASE WHEN $sort = 'certified' THEN NOT certified END ASC,
  CASE WHEN $sort = 'rating' THEN score_note END DESC,
  distance_m ASC,
  id ASC
)
```

La construction repose sur une propriété du langage : dans une branche inactive, l'expression conditionnelle vaut la valeur nulle pour toutes les lignes. Elle ne discrimine donc rien, et le tri retombe sur la clé suivante. Un seul plan de requête couvre ainsi les trois tris, sans qu'il soit nécessaire de concaténer du SQL — donc sans surface d'injection.

La note utilisée pour le tri par qualité n'est pas une moyenne mais une note bayésienne. Une moyenne simple laisserait un garage noté cinq sur deux avis complaisants devancer un garage noté quatre virgule six sur cent vingt-huit avis, ce qui est exactement ce qu'un tri « mieux noté » ne doit pas faire.

Pondération bayésienne de la note d'un garage

```
score_note = (n / (n + 20)) × note + (20 / (n + 20)) × 3,8
```

où n est le nombre d'avis, 20 le seuil à partir duquel la note pèse pleinement, et 3,8 la note moyenne de référence.

Enfin, le filtre par service utilise deux opérateurs différents selon le contexte. La recherche libre exige que le garage offre tous les services demandés ; la recherche déclenchée par un appel d'urgence se contente d'un service au moins. Les deux consomment le même index, la bascule ne coûte donc rien en performance, mais elle change complètement le sens — et c'est

bien le second sens dont l'appel d'urgence a besoin, puisque la taxonomie lui fournit des compétences alternatives et non une liste d'exigences.

5.7 Contrats de l'interface de programmation

L'interface expose quarante-quatre routes réparties en six domaines. Les erreurs suivent toujours la même forme, un objet portant un code, un message et éventuellement les champs fautifs ; le mobile traduit sur le code et jamais sur le message, ce qui permet de changer les libellés sans casser les clients. Les dates sont exprimées en temps universel au format normalisé, la pagination est enveloppée, et chaque entrée est validée par un schéma partagé avant d'atteindre le contrôleur.

Tableau 7 — Routes principales de l'interface de programmation

Méthode	Route	Rôle
POST	/auth/signup · /login · /refresh · /logout	Authentification, rotation du jeton de rafraîchissement, jeton opaque haché en base
GET	/garages/nearby	Recherche de proximité, publique, paramètres de rayon, tri, services et ouverture
GET	/garages/:id · /:id/reviews	Fiche complète et avis paginés
POST	/requests	Crée la demande et renvoie les garages classés dans la même réponse
GET	/requests/active · /mine · /garage	Demande en cours, historique du client, file de travail du garage
POST	/requests/:id/select · /accept · /decline · /en-route	Transitions de la machine à états
POST	/requests/:id/arrive	Confirmation d'arrivée, appelée par les deux parties, idempotente
POST	/requests/:id/cancel	Annulation par le client
GET	/requests/:id	État complet et dernier numéro de séquence — route du mode dégradé
GET	/requests/:id/route · /approach	Tracé d'itinéraire et phase d'approche
POST	/requests/:id/review	Publication d'un avis, refusée si la demande n'est pas clôturée
GET · PATCH · DELETE	/me · /me/garage · /me/vehicles	Profil, garage du propriétaire, véhicules enregistrés
GET	/me/loyalty · /me/loyalty/history	Solde, grade et historique des mouvements
POST	/me/referral/claim	Rattachement à un parrain

POST	/driving/sessions	Envoi groupé en fin de session, tolérant au hors-ligne
POST	/uploads/sign	Signature d'envoi direct vers le service de stockage

Deux routes appellent un commentaire.

La route d'état d'une demande est la clé de voûte du mode dégradé. Elle retourne l'état complet et le dernier numéro de séquence, ce qui suffit à reconstruire un écran de suivi depuis zéro. C'est délibérément la même route que celle du premier chargement.

La route d'envoi des sessions de conduite fonctionne par lot en fin de session plutôt qu'en continu. Les sessions sont stockées localement pendant leur déroulement et synchronisées à la fin, avec un identifiant généré par le mobile qui rend le renvoi idempotent si le réseau coupe pendant la synchronisation. Une session de conduite n'a aucune raison de consommer du réseau en direct.

5.8 Protocole temps réel

Le canal temps réel repose sur des salles nommées par identifiant de demande, rejointes après vérification du jeton d'authentification et contrôle d'appartenance à la demande.

Tableau 8 — Événements du canal temps réel

Sens	Événement	Charge utile
Client vers serveur	request:join	Identifiant de demande et dernier numéro de séquence connu
Client vers serveur	request:position	Position, vitesse, cap et précision
Client vers serveur	request:leave	Sortie de la salle
Serveur vers client	request:state	Statut, dernier numéro de séquence et événements manqués
Serveur vers client	request:tracking	Les deux temps d'arrivée estimés et l'horodatage d'émission
Serveur vers client	request:event	Événement unitaire du journal
Serveur vers client	request:error	Code d'erreur normalisé
Serveur vers propriétaire	garage:jobs	File de travail complète du garage

La cadence d'émission combine un intervalle et un seuil de déplacement : une émission au plus toutes les quatre secondes, et seulement au-delà de quinze mètres parcourus. Le calcul qui a conduit à ces valeurs est simple. Un relevé pèse environ cent vingt octets ; une intervention de vingt minutes revient donc à une trentaine de kilooctets. Sans le seuil de

déplacement, un véhicule arrêté dans un embouteillage émettrait trois cents relevés identiques pour rien.

Les positions reçues passent par un lissage exponentiel et par un rejet des déplacements impliquant plus de cent cinquante kilomètres par heure, ce qui élimine les sauts de mesure caractéristiques d'un signal réfléchi entre deux immeubles.

Enfin, l'indicateur de fraîcheur affiché à l'écran n'est pas l'intervalle nominal d'émission mais la différence entre l'heure courante et l'horodatage du dernier paquet reçu. La distinction est importante : le premier est une promesse, le second est une mesure. Sur un réseau irrégulier, les deux divergent souvent, et c'est précisément quand ils divergent que l'utilisateur a besoin de savoir laquelle des deux valeurs est vraie.

5.9 Dispositif anti-fraude

Ce dispositif est traité ici et non au chapitre des risques, parce qu'il fait partie des spécifications détaillées : ses seuils sont des constantes du paquet partagé, et ses effets sont inscrits dans le schéma.

Le problème est le suivant. La double confirmation d'arrivée prouve que deux personnes se sont mises d'accord. Elle ne prouve pas qu'une intervention a eu lieu. Deux comptes complices, un client et un garage tenus par la même personne, peuvent se confirmer mutuellement sans que rien ne se produise, et faire ainsi imprimer de la valeur au système.

La parade principale est la preuve de mouvement. Les relevés de position ne servent pas qu'à l'affichage : ils sont conservés et constituent une piste d'audit. Pour qu'un crédit soit validé, la trace doit montrer deux parties initialement éloignées puis convergentes. C'est ce qui casse la collusion statique, c'est-à-dire celle qui ne coûte rien à monter — deux téléphones posés côte à côte sur une table.

Tableau 9 — Seuils du dispositif anti-fraude

Garde-fou	Valeur	Ce qu'il empêche
Durée minimale d'intervention	180 secondes	Une intervention trop courte pour être plausible
Séparation initiale minimale	300 mètres	Deux téléphones côte à côte se confirmant mutuellement
Distance parcourue par le garagiste	200 mètres	Un garagiste qui n'a jamais bougé
Période de différé avant confirmation	24 heures	Ferme une fenêtre d'annulation en cas de détection
Plafond par paire client-garage	3 crédits sur 30 jours	Une paire complice qui répéterait l'opération
Seuil de revue manuelle	2 000 points	Une conversion importante sans contrôle humain

Ces seuils sont posés dès le schéma. Le raisonnement économique est explicite : installés dès le départ, ils ne coûtent rien ; rétro-installés après une fraude, ils coûtent une migration, un audit comptable et une perte de confiance. Enfin, la conversion en Mobile Money n'est jamais automatique dans la première version, ce qui laisse un humain dans la boucle sur l'opération irréversible.

Choix techniques et justifications

Ce chapitre justifie les décisions techniques une par une. Chacune est présentée avec l'alternative écartée et la raison concrète de l'écart, parce qu'un choix qui ne dit pas ce qu'il refuse n'est pas un choix.

6.1 Monorepo et contrats partagés

L'alternative naturelle était deux dépôts distincts, l'un pour le mobile, l'autre pour le serveur. C'est la structure la plus courante et la plus simple à mettre en place.

Elle a été écartée à cause d'un problème précis : trois choses ne doivent jamais diverger entre les deux extrémités. La taxonomie des pannes, parce que le serveur valide la paire véhicule-panne que le mobile a proposée. La forme des réponses, en particulier le rang, parce que le mobile l'affiche sans le recalculer. Et le barème de fidélité, parce que le serveur crédite avec et que le mobile affiche avec — toute divergence entre les deux serait une divergence comptable.

Dans deux dépôts séparés, ces trois choses existent en double. Elles restent synchronisées tant que quelqu'un y pense, et cessent de l'être un mardi après-midi. Le monorepo avec un paquet partagé transforme la divergence en erreur de compilation : elle est détectée avant même d'être commise.

Le coût est réel et il faut le nommer : le paquet partagé doit être compilé avant les deux autres, ce qui ajoute une étape obligatoire à toute installation et une source de confusion pour quelqu'un qui découvre le projet.

6.2 PostgreSQL avec extension géospatiale

L'alternative était une base documentaire avec un index géospatial, qui aurait été un choix défendable et fréquent pour ce type de projet.

Le facteur déterminant a été le besoin de contraintes. Les règles métier de la fidélité doivent être inscrites dans le schéma pour tenir dans la durée, et les contraintes de vérification, les index d'unicité partiels et les contraintes référentielles n'ont pas d'équivalent robuste dans une base documentaire. Le second facteur a été la qualité du support géospatial : la fonction de filtrage par distance combinée à l'index adéquat produit un plan d'exécution que l'on peut vérifier, mesurer et tester, ce qui a permis d'écrire un test automatisé sur le plan lui-même.

Le type géographique a été préféré au type géométrique parce qu'il rend directement des mètres en tenant compte de la courbure terrestre, sans conversion ni choix de projection à justifier.

L'hébergement retenu est une offre serverless qui supporte l'extension nativement et propose des branches de base, ce qui permet d'avoir une branche de développement et une branche de test distinctes de la production sans multiplier les instances.

6.3 Constructeur de requêtes typé plutôt qu'ORM complet

L'alternative était un ORM à part entière, avec génération de migrations et modèle déclaratif.

Le problème est que la moitié du produit est géospatiale et que les ORM courants ne savent pas typer les colonnes géographiques. Le contournement habituel consiste à déclarer ces colonnes comme non supportées, à passer par des requêtes brutes pour tout ce qui touche à la géométrie, et à éditer à la main les migrations générées. Cela revient à abandonner la sûreté de typage exactement là où elle sert le plus, c'est-à-dire sur le cœur du produit.

Tableau 10 — Comparaison sur la partie géospatiale

Critère	ORM avec colonnes non supportées	Constructeur de requêtes typé
Requête géospatiale	Requête brute, résultat non typé	Fragment SQL, résultat typé
Migrations	Générées puis éditées à la main	Fichiers SQL écrits directement
Dérive entre schéma et code	Invisible jusqu'à l'exécution	Erreur de compilation
Lisibilité du SQL produit	Indirecte	Directe
Courbe d'apprentissage	Plus douce	Plus raide, SQL requis

Le choix a été accompagné d'une précaution volontairement désagréable à l'usage : le type qui représente un point géographique n'accepte, en écriture, qu'un fragment SQL produit par une fonction dédiée, et rend en lecture une chaîne binaire inexploitable. On est donc obligé de passer par la fonction de construction et de projeter explicitement les coordonnées à la lecture. C'est pénible, et c'est précisément le but : on ne peut pas se tromper par distraction.

6.4 Rendu cartographique

Trois options ont été examinées : la solution cartographique de Google, celle d'Apple par l'intermédiaire du composant natif de la plateforme, et une bibliothèque de rendu vectoriel libre alimentée par un fournisseur de tuiles.

Le critère qui a tranché est la stylisation. L'identité visuelle du produit repose sur une palette chaude précise, et la carte occupe la moitié de l'écran principal. Or la solution de Google n'expose son style qu'une seule des deux plateformes, et la solution d'Apple ne se style pratiquement pas. On obtiendrait deux applications qui ne se ressemblent pas selon le téléphone, ce qui est intenable quand l'identité visuelle constitue le principal facteur de différenciation. La bibliothèque de rendu vectoriel libre produit le même style au pixel près sur les deux plateformes.

Le second critère est la donnée routière. Le fond cartographique libre est compétitif à Yaoundé grâce au travail de cartographie humanitaire, et il est meilleur que les fonds commerciaux sur les voies non bitumées — qui sont exactement celles où l'on tombe en panne. Les fonds commerciaux gardent l'avantage sur les points d'intérêt marchands, c'est-à-dire précisément la donnée que GeoCras produit lui-même et n'a donc pas à emprunter.

Le troisième critère est le coût. Le fournisseur retenu facture à l'usage, avec un palier gratuit généreux, là où son concurrent principal facture à l'utilisateur actif mensuel — un modèle qui pénalise une application dont chaque utilisateur ouvre la carte rarement mais dont la base est large. Une porte de sortie existe si le coût devient un problème : l'auto-hébergement des tuiles ne change qu'une adresse.

Le risque assumé est que le support de la nouvelle architecture de la plateforme mobile n'est pas confirmé pour cette bibliothèque. C'est le troisième risque du projet, et deux issues de secours sont documentées à l'avance : désactiver la nouvelle architecture, ce que la version courante du kit de développement autorise encore et les suivantes non — c'est donc une dette avec une date de péremption —, ou se replier sur le composant natif en acceptant le compromis de style sur une des deux plateformes.

6.5 Canal temps réel

Trois options : une interrogation périodique, un flux unidirectionnel envoyé par le serveur, ou une bibliothèque de socket bidirectionnelle.

L'interrogation périodique a été écartée comme mode principal : à quinze secondes d'intervalle, un véhicule en mouvement devient inutilisable à suivre. Elle est en revanche conservée comme mode de repli, ce qui est un usage entièrement différent.

Le flux unidirectionnel a été écarté parce qu'il ne va que dans un sens, alors que le client doit émettre sa position. Il aurait fallu le doubler d'une voie montante, ce qui revient à réimplémenter à la main ce que la troisième option fournit.

Le socket brut a été écarté pour une raison qui tient entièrement au contexte : il aurait fallu réécrire la logique de reconnexion, et surtout renoncer au repli automatique sur une technique de transport plus tolérante quand le socket ne s'établit pas. Or c'est exactement ce repli qui sauve une session sur un réseau qui coupe. La bibliothèque retenue le fournit gratuitement.

6.6 React Native et Expo

L'alternative était un développement natif pour chaque plateforme.

Le facteur décisif est la cohérence de la stack. L'ensemble du projet est en TypeScript, du contrat partagé au serveur en passant par les écrans ; un développement natif aurait imposé deux langages supplémentaires et, surtout, aurait rendu impossible le partage des contrats.

Le second facteur est le coût de développement, qui n'est pas un argument secondaire pour un projet mené par une équipe réduite.

Le choix d'Expo par-dessus React Native ajoute l'accès aux capteurs, aux polices, au stockage sécurisé et aux notifications sans configuration native, ainsi qu'un service de construction distante qui permet de produire une version installable Android depuis n'importe quel poste, sans chaîne de compilation locale ni machine Apple. Sur un projet développé depuis Yaoundé, ce dernier point n'est pas anecdotique.

6.7 Répartition de l'état côté mobile

Le choix de l'outil est déterminé par la fréquence de changement de la donnée, et non par sa nature.

Tableau 11 — Répartition de l'état applicatif mobile

Nature de la donnée	Outil retenu	Raison
Cache des réponses serveur	Bibliothèque de cache de requêtes	Réessai, durée de fraîcheur et persistance hors ligne — décisifs sur réseau irrégulier
Session et jetons	Contexte et stockage sécurisé	Petit, rare, doit survivre au redémarrage
Suivi en temps réel	Magasin d'état léger	Change trois à cinq fois par seconde ; dans le cache de requêtes, on invaliderait en boucle
Mode conduite	Magasin léger et moteur hors du cycle de rendu	La source d'alertes est un émetteur pur, lu par sélecteurs
Préférences	Contexte et stockage asynchrone	Lu partout, modifié une fois par mois

Deux règles fermes en découlent, et elles sont respectées sans exception dans le code : aucune position temps réel ne transite par le cache de requêtes, et le thème reste dans un contexte.

6.8 Stockage des photographies

Les photographies de panne et de garage sont envoyées directement du téléphone vers un service de stockage tiers, au moyen d'une signature délivrée par le serveur. Elles ne transitent jamais par le serveur applicatif.

La raison est double. D'abord la charge : faire transiter des images par un serveur applicatif consomme sa bande passante et sa mémoire pour une opération qui n'a aucune valeur métier. Ensuite la fiabilité en réseau dégradé : un envoi direct peut être repris sans repasser par toute la chaîne applicative. La signature délivrée par le serveur contraint le pré-réglage d'envoi, ce qui évite qu'un jeton dérobé ne permette de déposer n'importe quoi.

6.9 Synthèse des choix

Tableau 12 — Synthèse des choix techniques et de leur justification principale

Domaine	Choix	Justification déterminante
Organisation du code	Monorepo avec paquet partagé	Rend la divergence des contrats impossible à compiler
Base de données	PostgreSQL avec extension géospatiale	Contraintes métier dans le schéma et plan de requête vérifiable
Accès aux données	Constructeur de requêtes typé	Conserve le typage sur la moitié géospatiale du produit
Hébergement de la base	Offre serverless avec branches	Branches de développement et de test isolées de la production
Client mobile	React Native et Expo	Une seule stack TypeScript ; construction distante sans machine Apple
Cartographie	Rendu vectoriel libre et tuiles à l'usage	Style identique sur les deux plateformes ; meilleure couverture des voies non bitumées
Temps réel	Socket bidirectionnel avec repli de transport	Le repli est ce qui sauve une session sur réseau qui coupe
Itinéraire	Moteur auto-hébergé, à venir	Un lien vers une carte tierce ferait sortir de l'application et détruirait la preuve
Photographies	Envoi direct signé vers un service tiers	Ne charge pas le serveur applicatif ; reprise possible
Validation	Schémas partagés serveur et mobile	Une règle écrite une fois, appliquée aux deux extrémités

Résultats obtenus

7.1 État d'avancement

L'application est construite. L'ossature complète — navigation, authentification, carte, déclaration de panne, résultats classés, suivi temps réel, clôture, fidélité, mode conduite, espace garagiste — est en place et fonctionne de bout en bout.

Tableau 13 — Volumétrie du code livré

Indicateur	Valeur
Lignes de TypeScript et TSX	environ 47 500
Routes d'écran mobile	26
Écrans dont le code dépasse 400 lignes	15
Routes exposées par l'interface de programmation	44
Tables de base de données	14
Migrations SQL versionnées	7
Fichiers de tests automatisés	26
Garages du jeu de données de développement	32, à Yaoundé

Un point doit être signalé avec netteté avant la présentation des interfaces. Les visuels qui suivent sont les maquettes de référence du produit, celles qui ont été dessinées avant le développement et qui ont servi de contrat visuel tout au long de sa réalisation. Ce ne sont pas des captures prises sur un appareil en fonctionnement. La raison en est que la bibliothèque de rendu cartographique ne fonctionne pas dans l'environnement de développement rapide et exige une version installable construite spécifiquement, ce qui suppose un appareil physique. Les captures réelles seront produites à l'occasion de la campagne d'essais sur appareil décrite en fin de chapitre. On notera par ailleurs que les libellés de quartiers visibles sur ces maquettes sont ceux de Douala, ville initialement retenue ; le produit se lance à Yaoundé et le jeu de données de développement l'est déjà.

7.2 Les interfaces

7.2.1 Lancement et carte principale

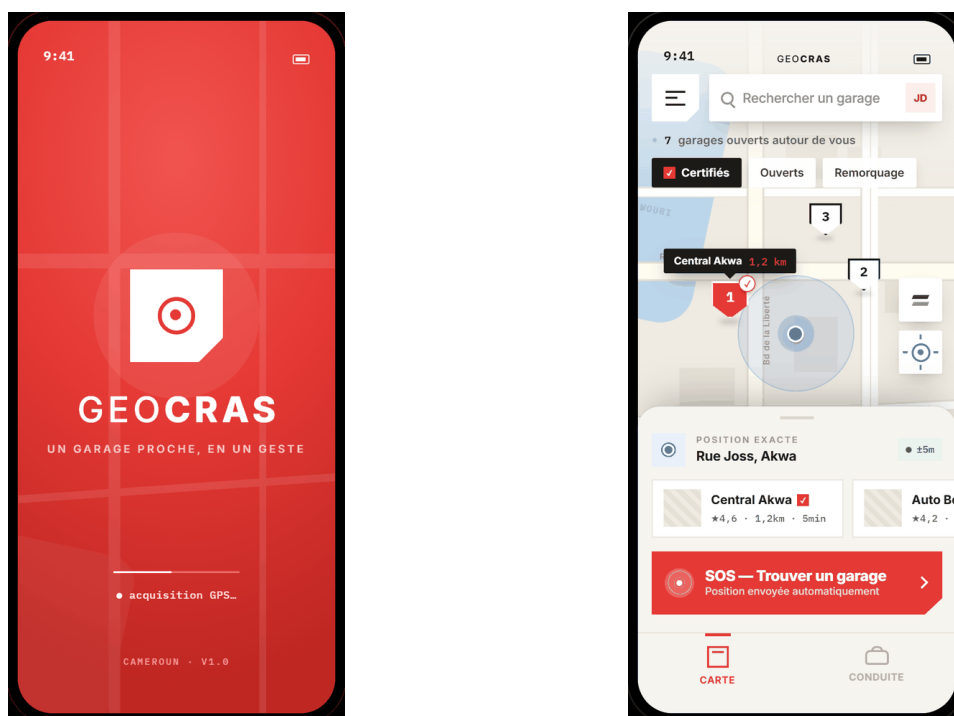


Figure 9 — Écran de lancement avec acquisition de la position, et carte principale avec marqueurs numérotés

L'écran de lancement n'est pas décoratif : il attend le résultat réel de l'acquisition de position. La durée d'affichage minimale qui lui est imposée est un plancher et non un plafond — l'écran ne disparaît jamais avant que la position soit résolue, on empêche seulement qu'un signal obtenu en cent vingt millisecondes produise un clignotement illisible. Si la position ne peut pas être obtenue, l'application s'ouvre quand même : rester bloqué priverait de l'application quelqu'un qui a refusé la permission.

La carte principale porte l'essentiel de l'identité visuelle. Les marqueurs sont des écussons pentagonaux numérotés et non les gouttes par défaut ; un garage certifié est rempli en rouge avec une pastille de validation, un garage non certifié reste blanc avec une bordure d'encre. Le numéro affiché est celui calculé par le serveur. Le bouton d'urgence est l'élément le plus visible de l'écran, ce qui découle directement de l'analyse de la situation d'usage : l'utilisateur est dehors, souvent sous stress, et n'a pas à chercher.

7.2.2 Déclaration de panne et résultats

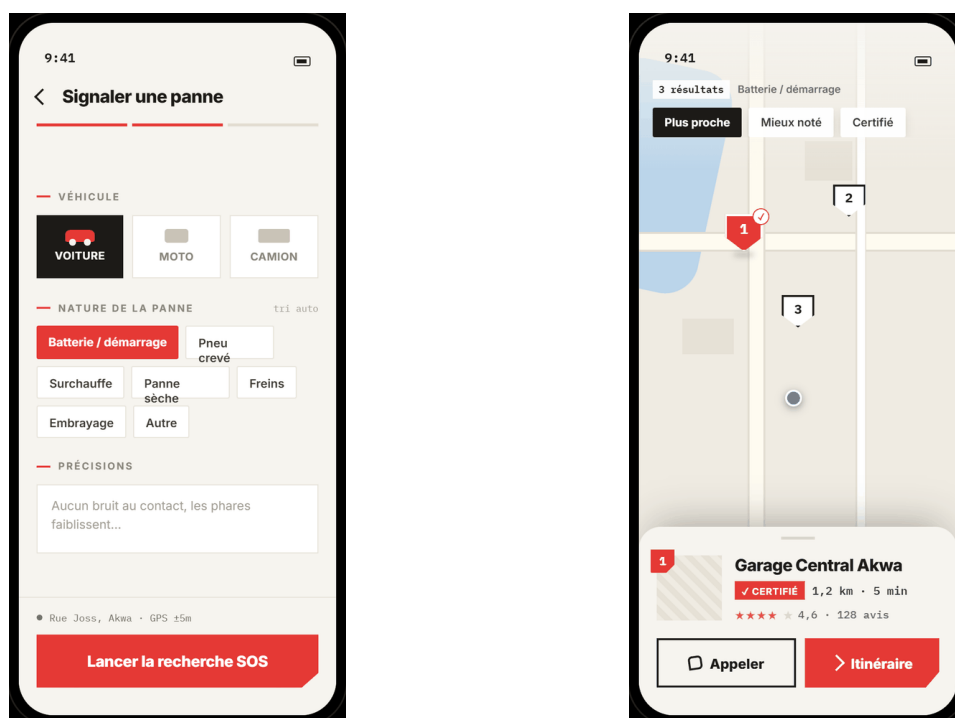


Figure 10 — Déclaration de panne avec tri automatique des problèmes, et résultats classés avec fiche contextuelle

La déclaration de panne est l'écran le plus travaillé de l'application, et son code est le plus volumineux du projet. Le tri automatique visible ici n'est pas un effet d'interface : la liste des problèmes proposés est celle du paquet partagé, ordonnée par véhicule, la panne la plus probable en tête. Choisir une moto ne fait pas seulement disparaître les problèmes de boîte de vitesses, cela remonte la transmission par chaîne et le carburateur.

L'écran de résultats affiche la liste et la carte simultanément, avec la même numérotation. Changer de tri renumérote les marqueurs et la liste ensemble, parce que les deux lisent le même rang venu du serveur. La fiche contextuelle qui s'ouvre au toucher d'un marqueur porte les deux actions qui comptent dans la situation : appeler et calculer l'itinéraire.

7.2.3 Itinéraire, suivi et avis

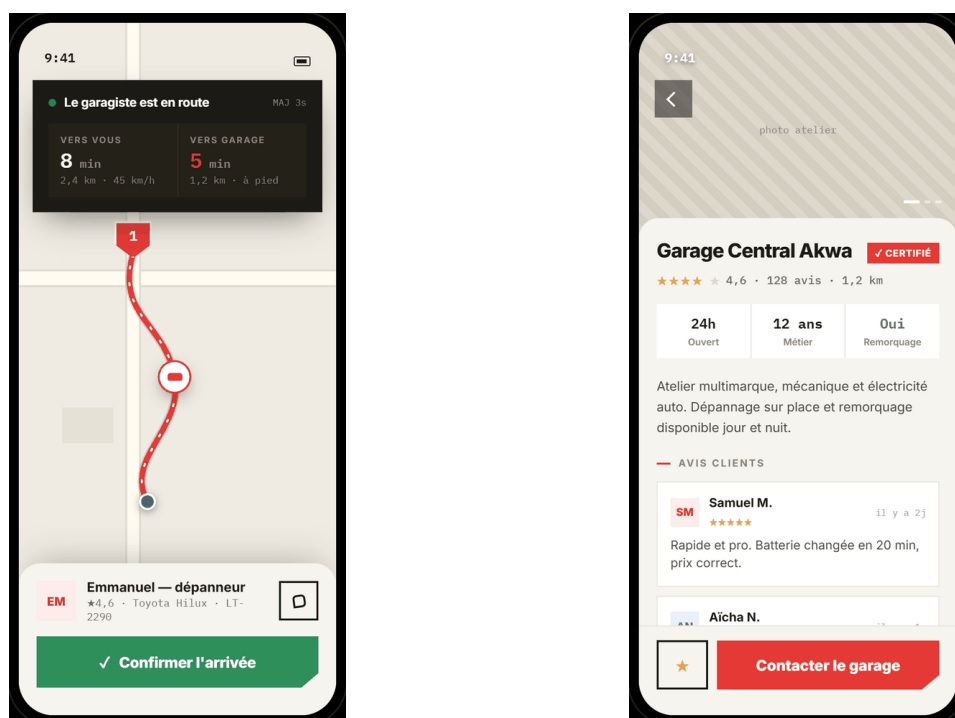


Figure 11 — Suivi bidirectionnel avec double estimation d'arrivée, et fiche complète d'un garage avec ses avis

L'écran de suivi affiche les deux temps d'arrivée estimés côte à côte, celui du garagiste vers le client et celui du client vers le garage. Les deux sont calculés par le serveur et identiques des deux côtés. L'indicateur de fraîcheur visible dans le bandeau affiche l'âge réel de la dernière donnée reçue, calculé par différence avec l'horodatage d'émission, et non l'intervalle nominal.

La fiche complète d'un garage présente ses photographies, ses services, ses horaires et ses avis. La note affichée est la note brute ; la note pondérée qui sert au classement n'est pas montrée, parce qu'elle serait incompréhensible pour un utilisateur et que son rôle est de trier, pas d'informer.

7.2.4 Mode conduite

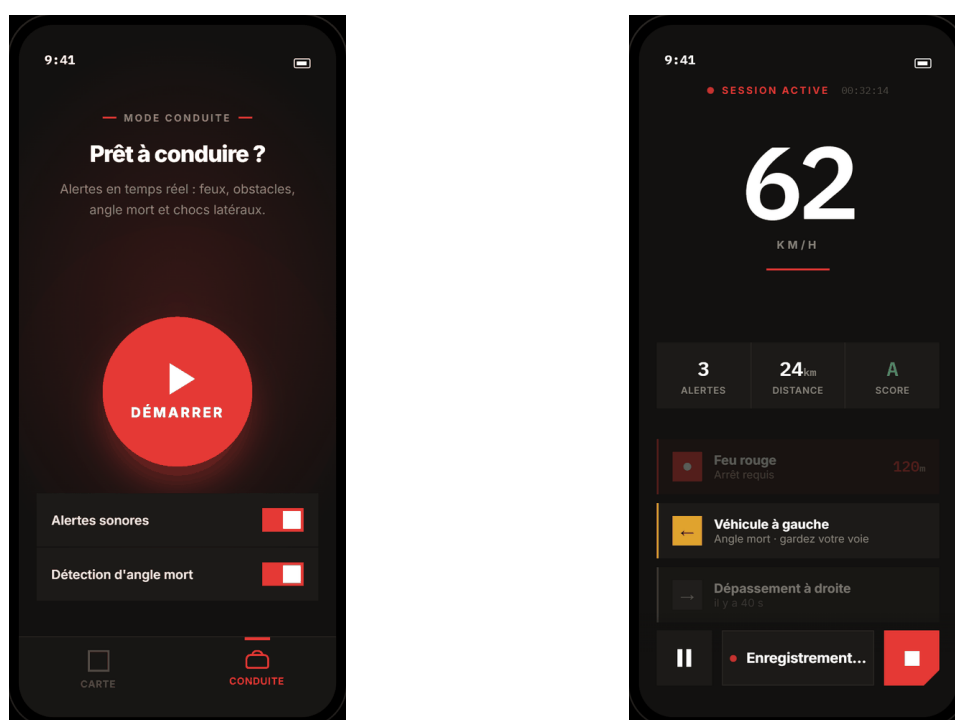


Figure 12 — Mode conduite au repos et en session, avec vitesse, statistiques et pile d'alertes

Le mode conduite affiche la vitesse en grand caractère monospace, les statistiques de la session en cours et une pile d'alertes horodatées. Les alertes sont produites par une source d'événements simulée, isolée derrière une interface qui définit un contrat minimal : démarrer, arrêter, émettre. Une source réelle branchée sur des capteurs implémenterait la même interface et ne demanderait aucune modification des écrans.

Les sessions sont enregistrées localement pendant leur déroulement et envoyées groupées à la fin, avec un identifiant généré par le mobile qui rend le renvoi idempotent. Une session de conduite ne consomme donc pas de réseau en direct.

7.2.5 Fidélité, menu et paramètres

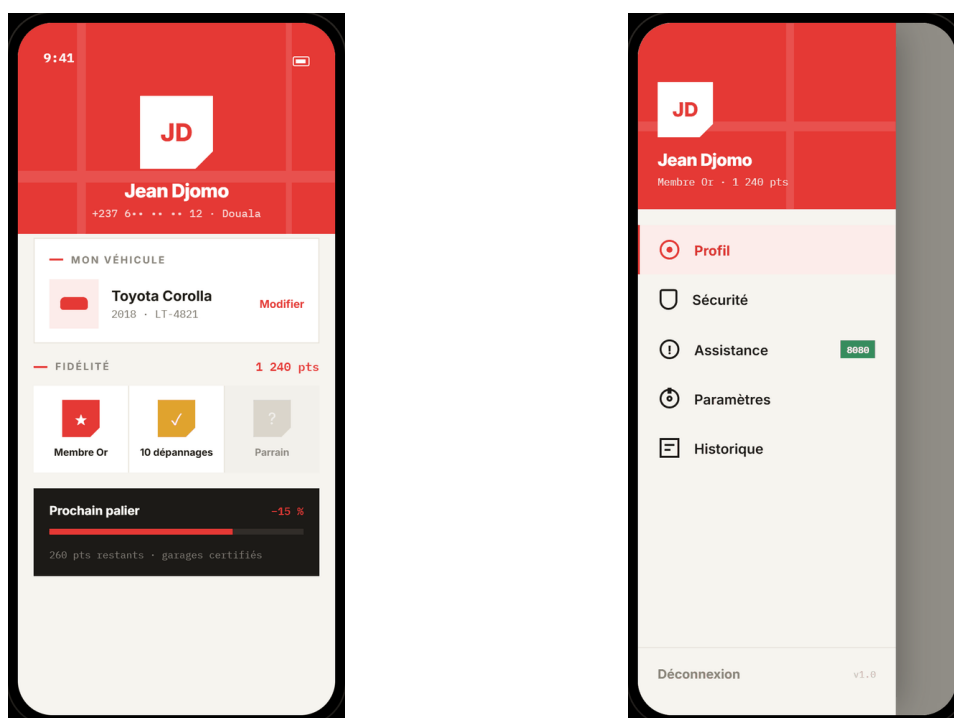


Figure 13 — Espace de fidélité avec grade et progression, et tiroir latéral de navigation

L'espace de fidélité présente le solde de points, le grade courant et la progression vers le suivant. La distinction entre les deux mesures y est rendue visible : le solde est une monnaie qui peut baisser, le grade une ancienneté qui ne baisse pas. La progression affichée est un compte de réparations terminées, pas un compte de points.

Le tiroir latéral donne accès au profil, à la sécurité, à l'assistance téléphonique, aux paramètres et à l'historique. Son en-tête affiche le grade et le solde en caractère monospacé, conformément à la règle qui veut que toute donnée mesurée le soit.

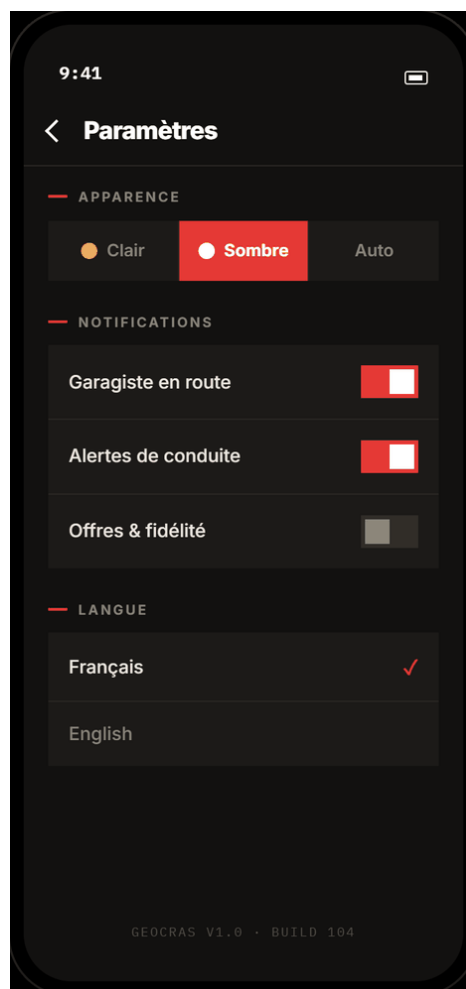


Figure 14 — Écran de paramètres en mode sombre

Le mode sombre n'est pas un simple inversement de contraste. Il conserve la chaleur de la palette — le fond sombre reste chaud plutôt que bleuté — parce que basculer sur des gris froids en mode sombre aurait produit deux applications différentes selon le réglage du téléphone. Toutes les couleurs des deux modes sont définies dans un fichier unique de jetons, et aucune couleur n'est écrite en dur dans un composant.

7.3 Tests et validation

La suite de tests compte cent trente-trois tests répartis en vingt-six fichiers.

Tableau 14 — Répartition des tests automatisés

Suite	Nombre	Base de données requise
Contrats partagés : barème, taxonomie, géométrie, machine à états	49	Non
Interface de programmation : routes, SQL compilé, calcul d'arrivée	35	Non
Interface de programmation :	21	Oui

géospatial, plan d'index, tris, filtres, repli		
Mobile : filtre de position, moteur de simulation, file de travail	28	Non

Les tests géospatiaux exigent une véritable instance PostgreSQL avec son extension. Aucune base en mémoire ne sait exécuter la fonction de filtrage par distance, et tester la requête centrale du produit contre une imitation ne prouverait rien. Ces tests sont donc ignorés par défaut et ne s'exécutent que si une base de test est explicitement fournie, sur une branche dédiée puisque la suite réinitialise le schéma.

Le test le plus intéressant du lot ne vérifie pas un résultat mais un plan d'exécution : il exige que la recherche de proximité passe par un parcours d'index géospatial et non par un parcours séquentiel. C'est une exigence non fonctionnelle transformée en test, et c'est la seule manière d'empêcher une régression de performance de passer inaperçue pendant six mois.

7.4 Ce qui reste à faire

Le document serait incomplet s'il ne disait pas ce qui n'est pas fait.

La campagne d'essais sur appareil est le point bloquant principal. La couche cartographique est écrite et les deux plateformes se compilent, mais le rendu réel ne se vérifie que sur un téléphone, avec une version installable construite pour l'occasion. Trois choses y seront mesurées : que la carte s'affiche avec les bonnes couleurs, que la nouvelle architecture de la plateforme ne provoque pas d'écran vide au montage, et que la fluidité tient soixante images par seconde avec vingt marqueurs sur un appareil de milieu de gamme. Les captures d'écran réelles seront produites à cette occasion et remplaceront les maquettes de ce chapitre.

Le moteur d'itinéraire auto-hébergé n'est pas déployé. Tant qu'il ne l'est pas, la durée de trajet reste approchée géométriquement et signalée comme provisoire dans l'interface.

Les notifications par poussée sont préparées côté routes mais non branchées. Il en va de même pour les liens profonds, dont l'infrastructure de navigation est prête.

L'écran de sécurité et l'interface anglaise restent à produire, cette dernière disposant déjà de tous ses libellés métier côté serveur.

Enfin, les seuils du dispositif anti-fraude sont posés dans le code mais n'ont jamais été confrontés à des données réelles. Ils devront être révisés après les premières semaines d'exploitation, ce qui est le propre de ce genre de valeur : elles se calibrent sur le terrain, pas sur une intuition.

Conclusion

8.1 Bilan

Le projet partait d'une observation simple : entre le moment où un véhicule s'immobilise à Yaoundé et celui où un professionnel se trouve à côté de lui, l'essentiel du temps perdu ne l'est pas à réparer, il l'est à trouver et à se trouver. GeoCras s'attaque à cette portion-là du problème, et l'application livrée le fait de bout en bout : la position remplace la description, le classement remplace la mémoire, le suivi remplace le rappel, et la trace remplace l'oubli.

Sur le plan technique, trois décisions structurent le résultat et paraissent, avec le recul, avoir été les bonnes. Le paquet de contrats partagés a supprimé une classe entière de défauts, ceux où les deux extrémités interprètent différemment la même donnée. L'inscription des règles métier sensibles dans le schéma de la base a produit des garanties qui ne dépendent d'aucune discipline de développement future. Et la conception du mode dégradé comme un mode de fonctionnement à part entière, plutôt que comme une gestion d'erreur, a donné une application qui reste honnête quand le réseau ne l'est plus.

Sur le plan de la méthode, le fait d'avoir dessiné les onze écrans avant d'écrire du code a eu des effets que l'on n'attendait pas au départ. Plusieurs décisions de modélisation en découlent directement — le calcul du rang en SQL, par exemple, est une exigence d'interface transformée en contrainte de requête. Une maquette qui fait autorité oblige à résoudre les problèmes dans le bon ordre.

8.2 Limites

Trois limites doivent être énoncées sans les atténuer.

La première est que le mode conduite ne mesure rien. Il est architecturé pour accueillir des capteurs réels, mais dans sa version actuelle il simule, et cette simulation ne doit jamais être présentée comme autre chose.

La deuxième est que le dispositif anti-fraude n'a pas été éprouvé. Il repose sur un raisonnement solide et sur des seuils argumentés, mais aucun de ces seuils n'a rencontré une tentative réelle. Il faut s'attendre à devoir les corriger.

La troisième est que l'application n'a pas encore tourné sur un appareil en conditions réelles, à Yaoundé, en réseau 3G, dans la rue. C'est la seule épreuve qui compte vraiment pour un produit dont toutes les hypothèses portent sur le terrain, et elle reste devant.

8.3 Perspectives

À court terme, la campagne d'essais sur appareil est prioritaire, suivie du déploiement du moteur d'itinéraire et du branchement des notifications.

À moyen terme, le service gagnerait un espace d'administration pour le traitement des dossiers de garage, aujourd'hui manuel, et une interface anglaise dont l'essentiel du travail est déjà fait côté serveur.

À plus long terme, deux directions ouvrent le produit. La première est la conversion des points en Mobile Money, qui suppose que le dispositif anti-fraude ait fait ses preuves sur des données réelles et qui restera assortie d'une revue humaine sur les montants significatifs. La seconde est l'extension à Douala puis aux autres villes, qui ne demande aucun travail d'ingénierie particulier — le modèle n'a rien de spécifique à Yaoundé — mais qui demande un travail de terrain considérable, puisqu'il faut constituer et vérifier une base de garages ville par ville. C'est d'ailleurs le vrai actif du produit : le code se réécrit, la base de garages vérifiés ne se réécrit pas.

8.4 Références

Tableau 15 — Ressources et documentation de référence

Domaine	Ressource
Extension géospatiale PostgreSQL	Documentation des fonctions de distance, des types géographiques et des index GIST
Plateforme mobile	Documentation versionnée du kit de développement employé
Rendu cartographique	Spécification de style vectoriel et documentation du greffon mobile
Fond cartographique	Données ouvertes collaboratives et politique d'usage des serveurs de tuiles
Temps réel	Documentation du protocole de socket bidirectionnel et de ses transports de repli
Validation	Documentation de la bibliothèque de schémas employée pour les contrats partagés
Modélisation	Spécification du langage de modélisation unifié, diagrammes de cas d'utilisation, de classes, d'états, de séquence et d'activité